



CMPT 413/713: Natural Language Processing

1

Classification - Logistic Regression

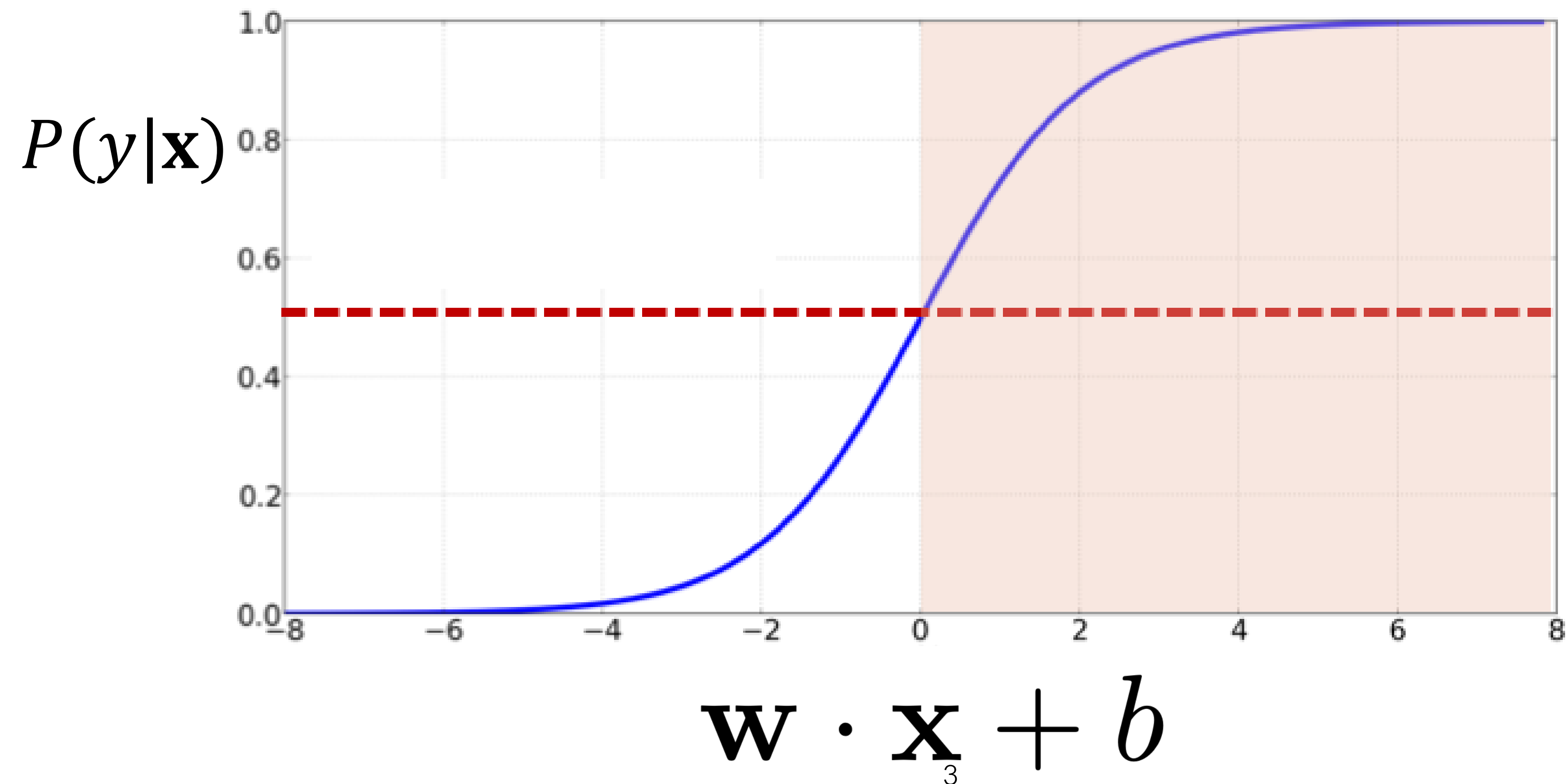
Zhiyun (Richard) Peng

2026-09-18

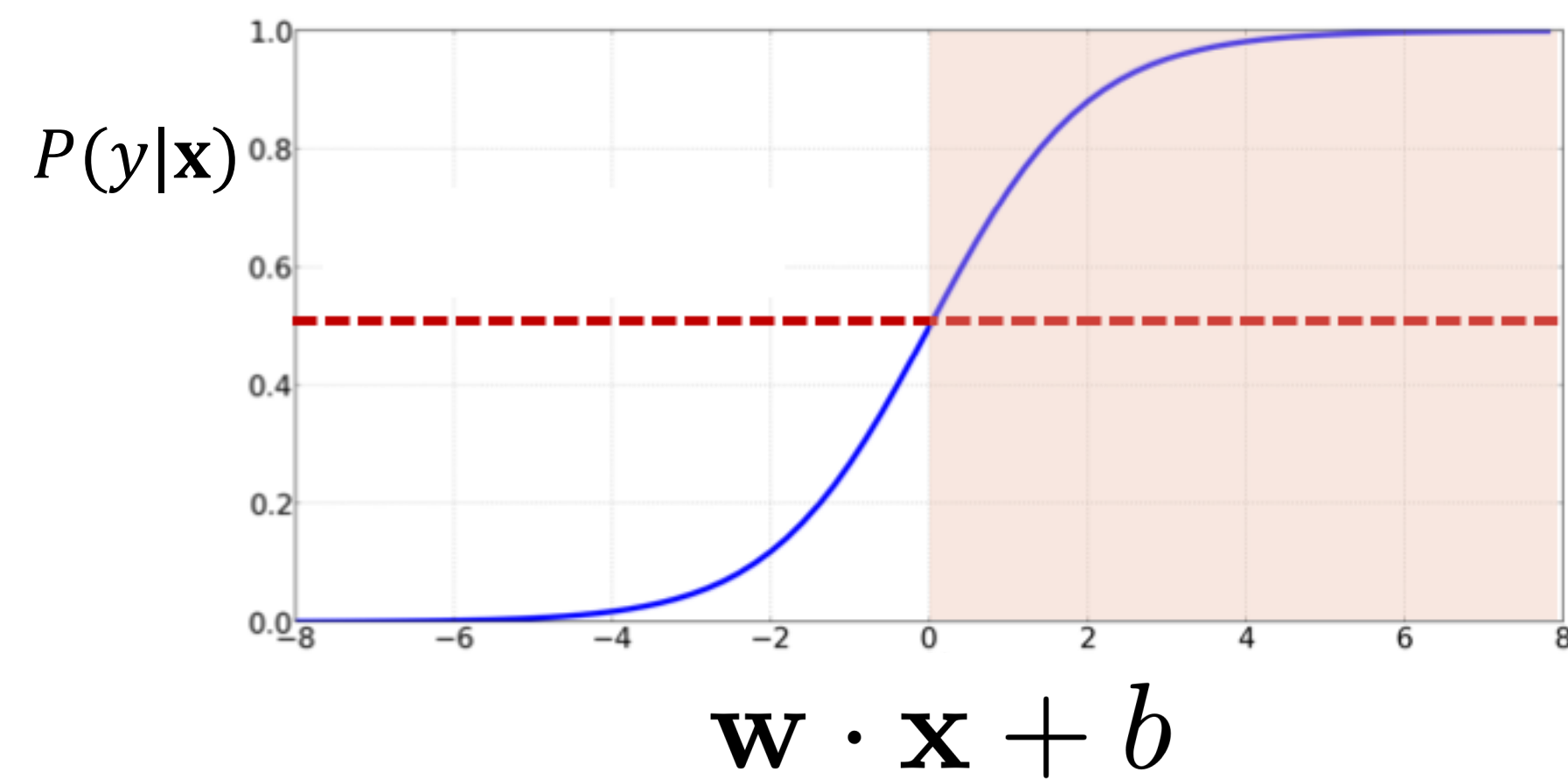
Logistic regression

Discriminative Model

- Logistic Regression: model $p(y|x)$ directly



Logistic Regression



- Powerful supervised model
- Baseline approach to most NLP tasks
- Connections with neural networks
- Binary (two classes) or multinomial (>2 classes)

Discriminative Model

discriminative model

- Logistic Regression:

$$\hat{c} = \operatorname{argmax}_c P(c|d)$$

A class

A sample

focus on
discriminating
features

- Naive Bayes:

generative model

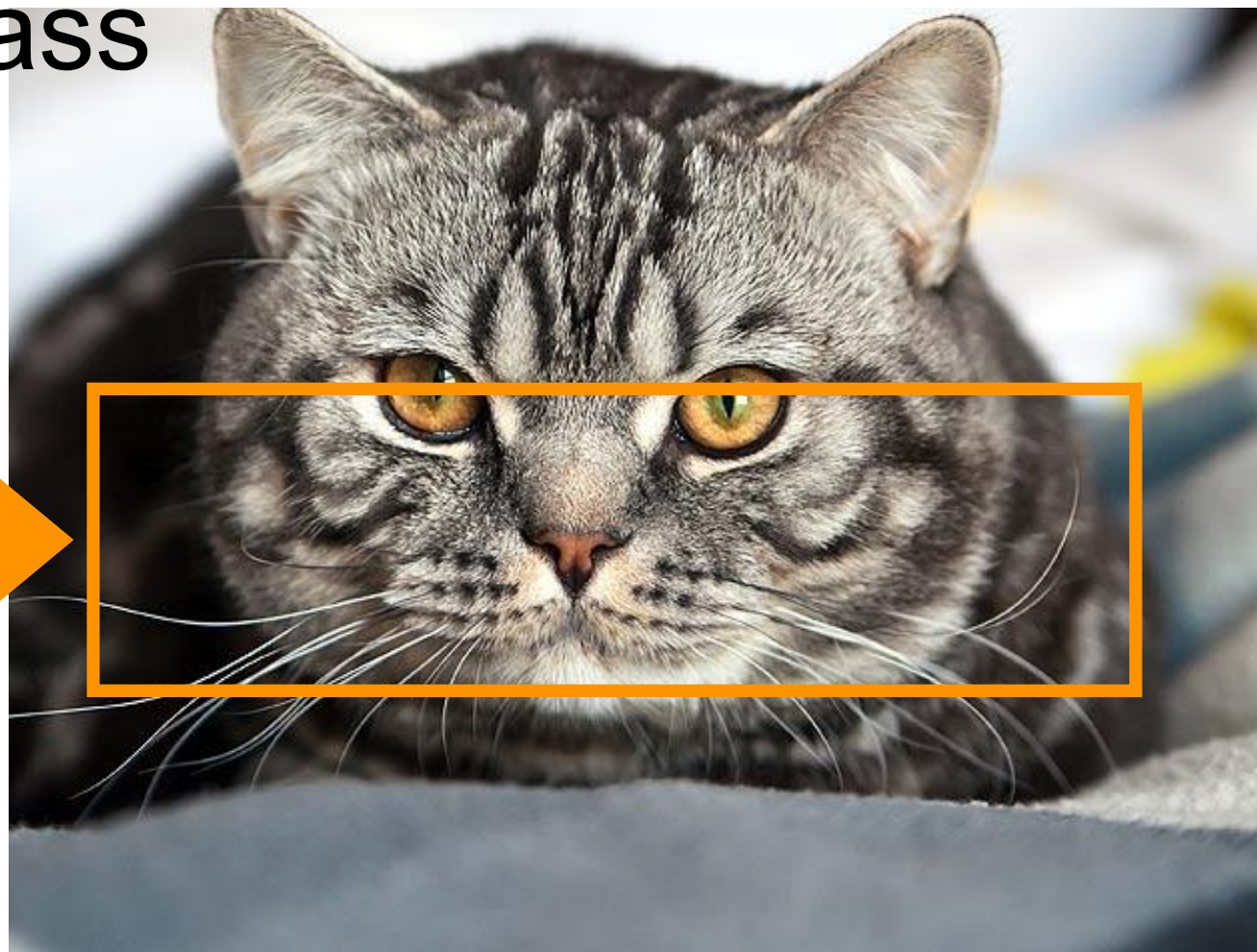
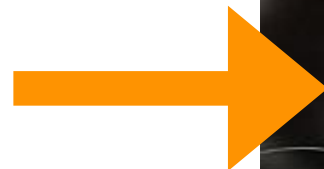
$$\hat{c} = \operatorname{argmax}_c P(c)P(d|c)$$

can be used to
generate the cat
and the dog

$$w_i \sim P(w | y)$$

A class

cats have
whiskers



Using Logistic Regression

- **Inputs:**
 1. Classification instance in a **feature representation**
 2. **Classification function** to compute $\hat{y}$ using $P(\hat{y}|x)$
 3. **Loss function** (for learning)
 4. Optimization **algorithm**

Training objective
- **Train phase:** Learn the **parameters** of the model to minimize **loss function**
- **Test phase:** Apply **parameters** to predict class given a new input x

Feature representation

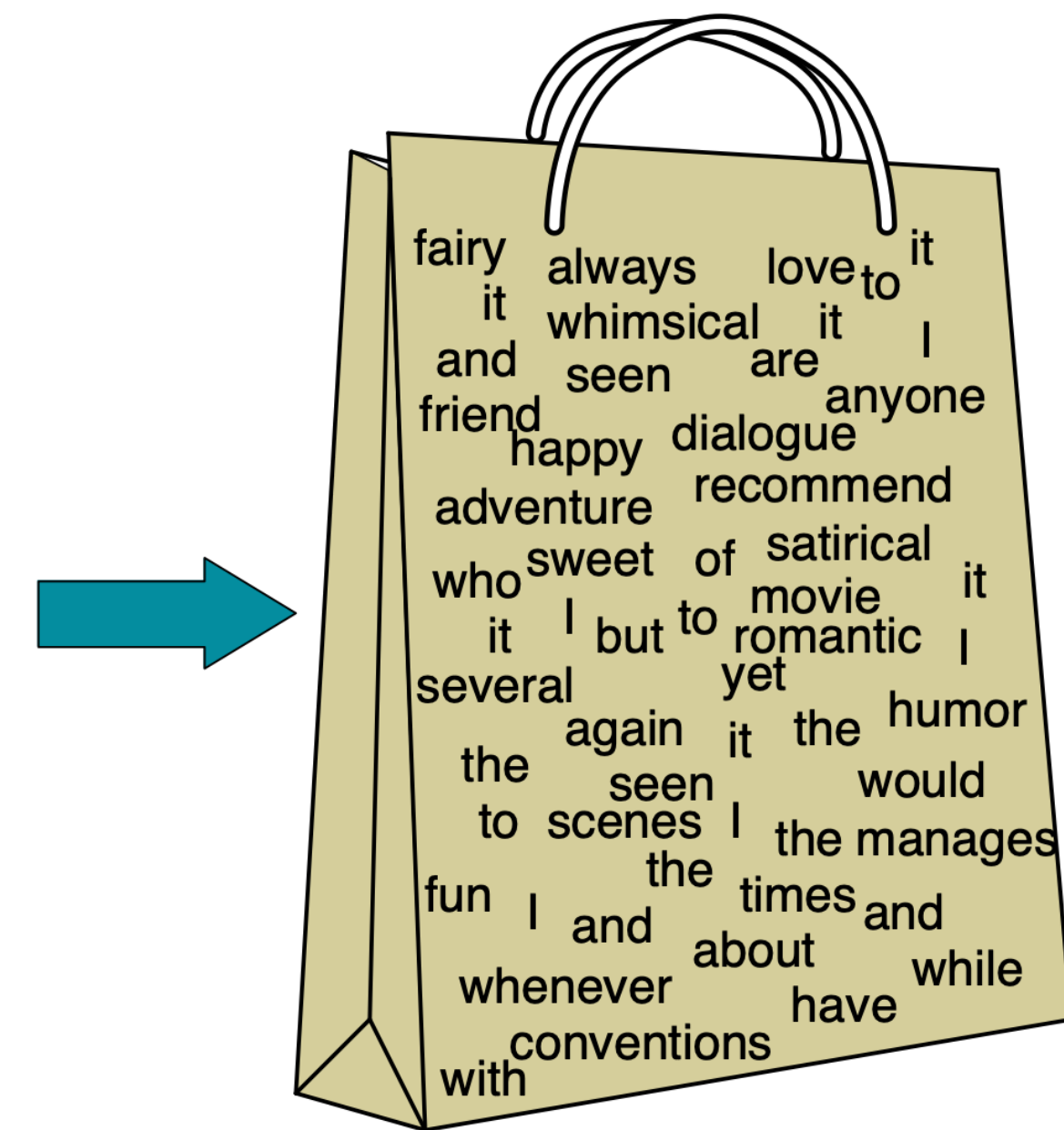
e.g. a document

- Input observation: $x^{(i)}$
- Feature vector: $[x_1, x_2, \dots, x_d]$
- Feature j of i^{th} input : $x_j^{(i)}$

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!

15

Bag of words



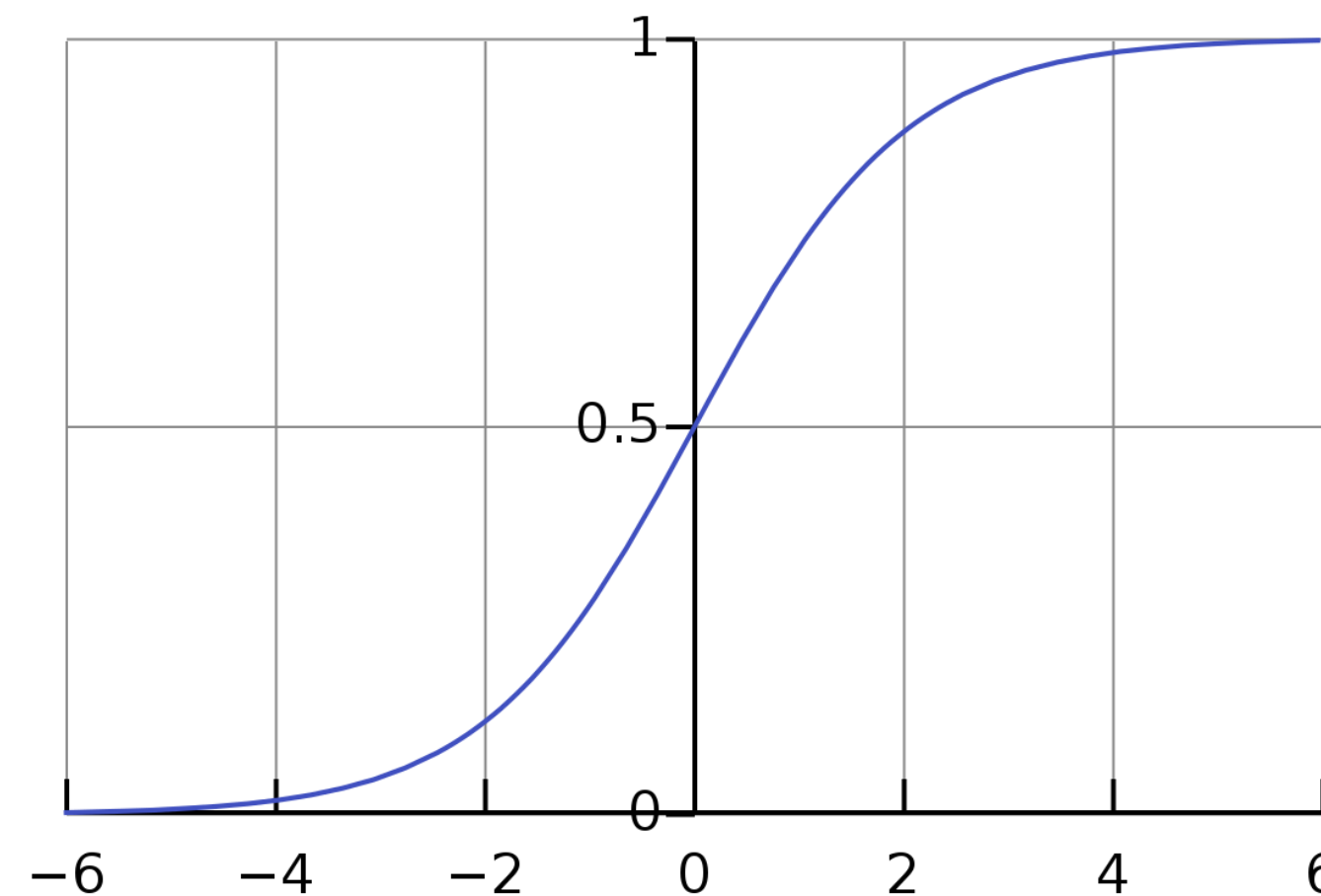
it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

Classification function

- *Given:* Input feature vector $[x_1, x_2, \dots, x_d]$
- *Output:* $P(y = 1|x)$ and $P(y = 0|x)$ *(binary classification)*
- Use a function, $F : \mathbb{R}^d \rightarrow [0, 1]$ to model the probability $P(y|x)$
- Sigmoid σ :

$$P(y|x) = \frac{1}{1 + e^{-z}}$$

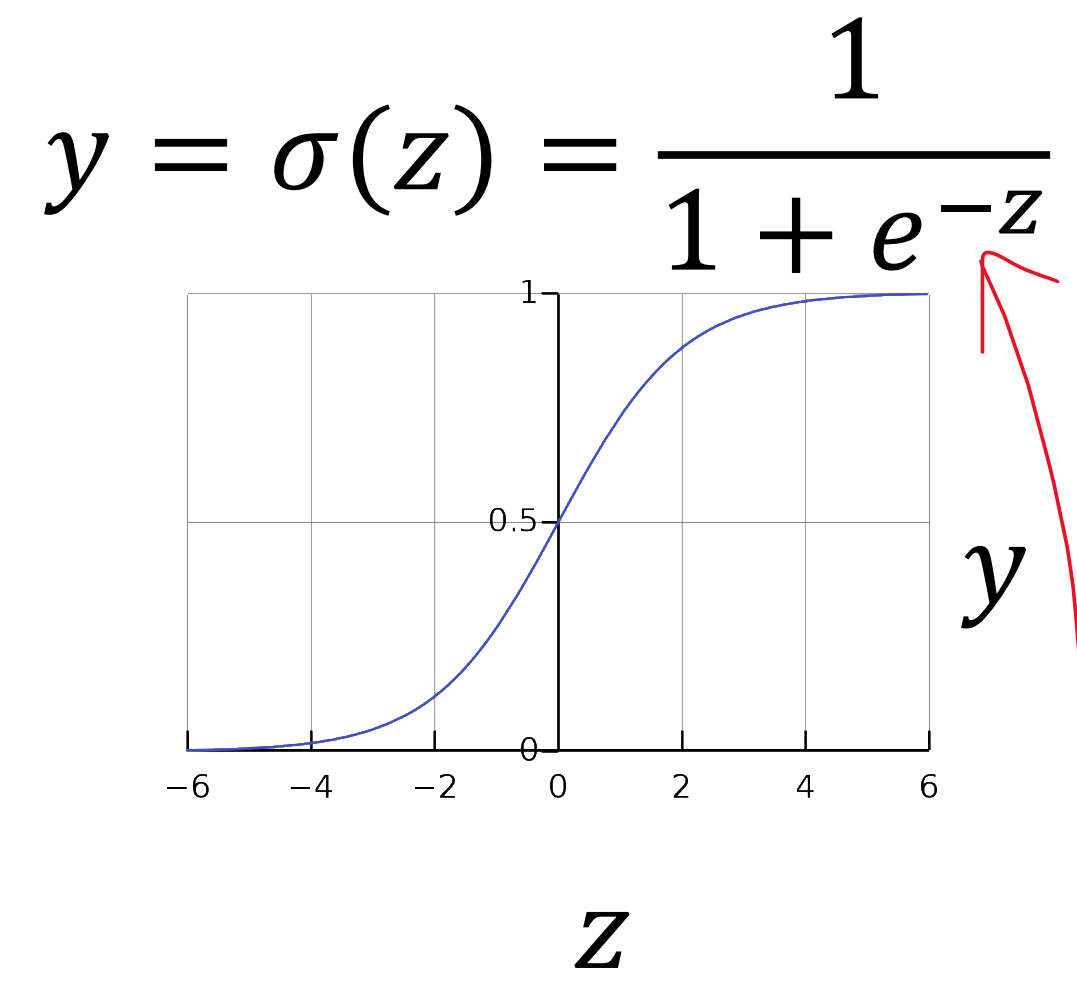
$$z = \sum_{i=1}^d w_i x_i + b$$



$P(y|x)$

If $p = 0.5$ (50/50 chance), Odds = $\frac{0.5}{0.5} = 1$ ("1 to 1").
 If $p = 0.8$ (80% chance), Odds = $\frac{0.8}{0.2} = 4$ ("4 to 1 in favor").
 If $p = 0.2$ (20% chance), Odds = $\frac{0.2}{0.8} = 0.25$ ("4 to 1 against").
 Range: $[0, \infty)$.

Why the sigmoid?



- Binary logistic regression uses the **log-odds (logit)** to determine the probability.

ℓ is the true log-odds:
 $\ell = \ln \left(\frac{P(y=1|x)}{P(y=0|x)} \right) = \ln \left(\frac{P(y=1|x)}{1 - P(y=1|x)} \right)$
SPAM
HAM

What is the probability p as a function of the log-odds?

- Log-odds:
 $\ell = \log \frac{p}{1-p}$

Value is also known as **logit (logistic unit)**

Natural logarithm $\ln$

$$\begin{aligned} \frac{p}{1-p} &= \exp(\ell) \\ p &= \exp(\ell)(1-p) \\ &= e^{\ell} - p \cdot e^{\ell} \\ \Rightarrow p + p \cdot e^{\ell} &= e^{\ell} \\ p(1 + \exp(\ell)) &= \exp(\ell) \\ p &= \frac{\exp(\ell)}{(1 + \exp(\ell))} \\ &= \frac{1}{(1 + \exp(-\ell))} = \sigma(\ell) \end{aligned}$$

Replace l with z

- z is a linear estimate of the log-odds ℓ : $z = \sum_{i=1}^d w_i x_i + b$

$z \approx \ell$

Weights and bias

- *Which features are important* and *how much*?

$$z = \sum_{i=1}^d w_i x_i + b$$

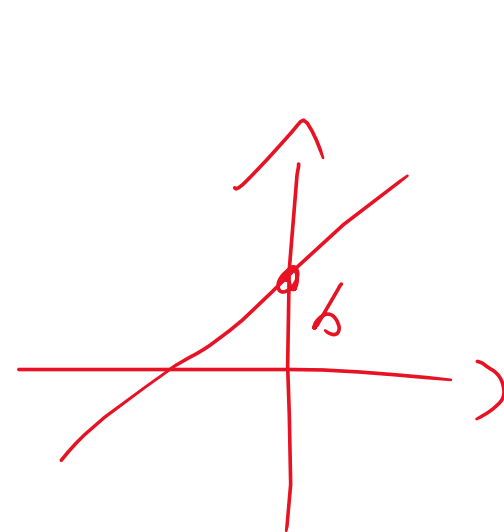
- Learn a vector of **weights** and a **bias**

- Weights: Vector of real numbers, $\mathbf{w} = [w_1, w_2, \dots, w_d]$

Vector form

$$z = \mathbf{w} \cdot \mathbf{x} + b = \mathbf{w}^T \mathbf{x} + b$$

- Bias: Scalar intercept, b



is the point where a line or curve crosses a coordinate axis on a graph

Putting it together

Given $\mathbf{x}$, compute $z = \mathbf{w} \cdot \mathbf{x} + b$

Compute probabilities: $P(y = 1|x) = \sigma(z) = \frac{1}{1+e^{-z}}$

$$P(y = 1) = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

$$\begin{aligned} P(y = 0) &= 1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b) \\ &= 1 - \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}} \\ &= \frac{e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}} \end{aligned}$$

Decision boundary:

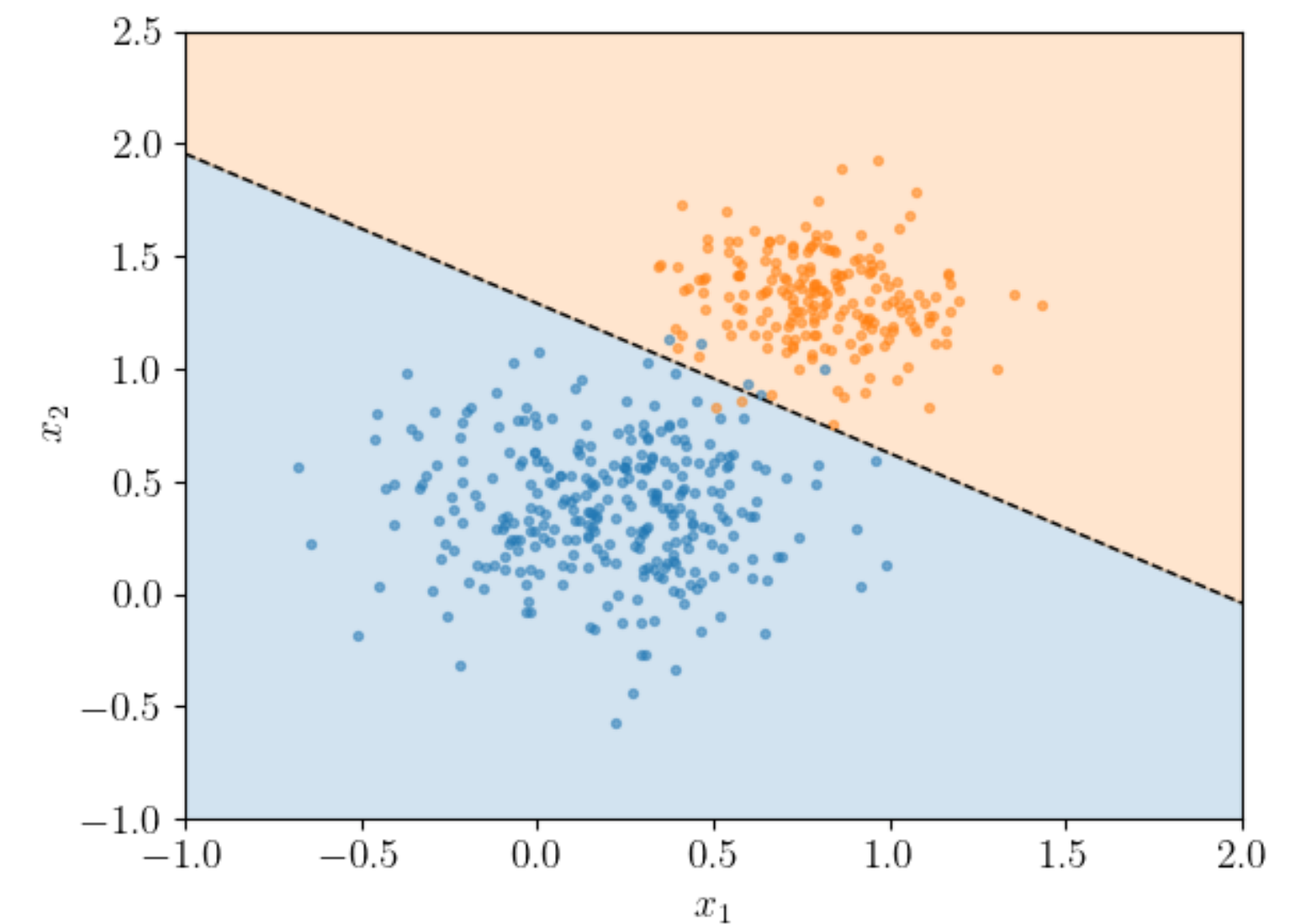
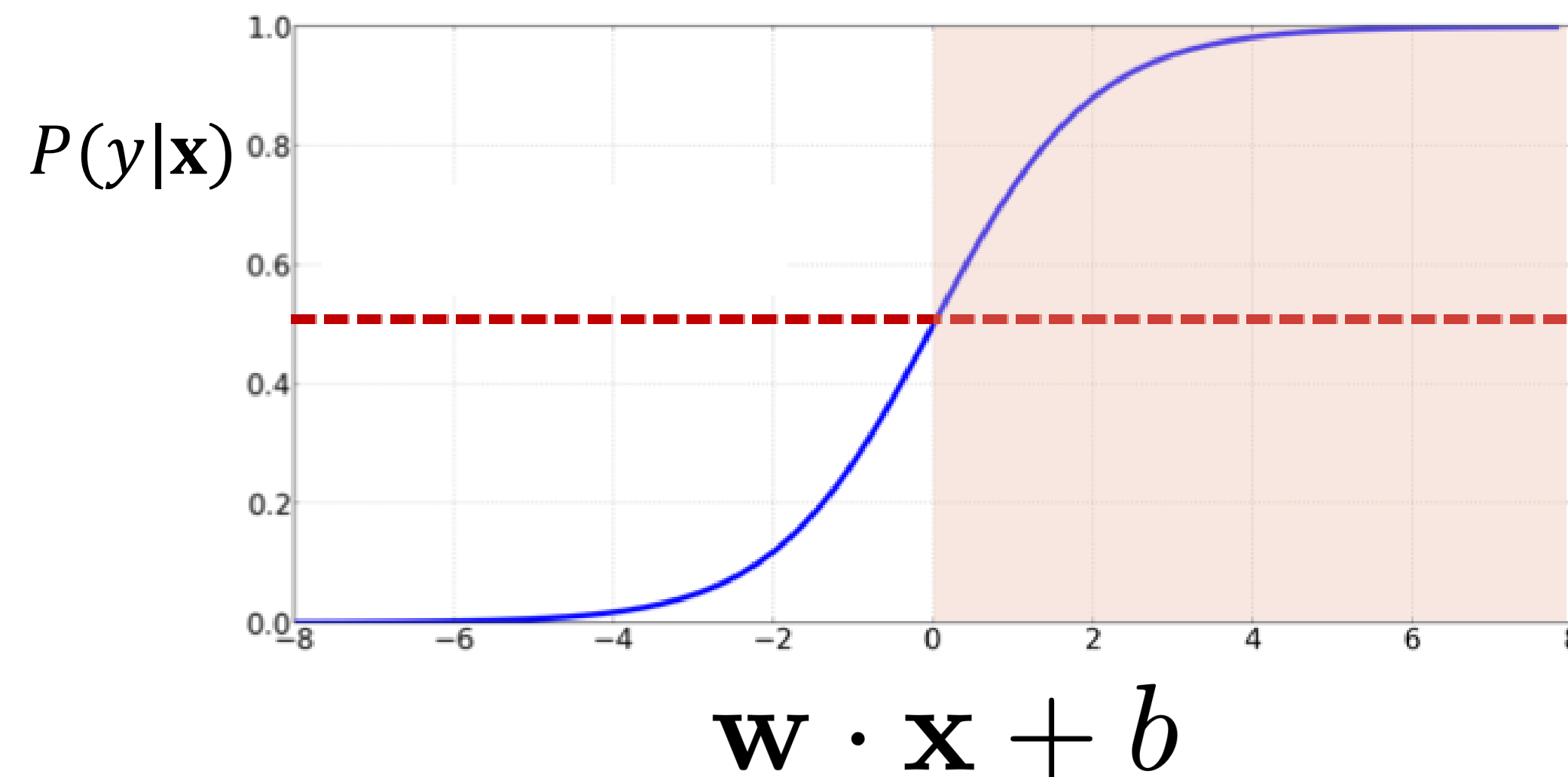
$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Decision boundary

Threshold can be set as
hyperparameter that is tuned

Decision boundary:

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases} \quad \begin{aligned} & \mathbf{w} \cdot \mathbf{x} + b > 0 \\ & \mathbf{w} \cdot \mathbf{x} + b < 0 \end{aligned}$$



Decision boundary is linear
function of features

What is the bias?

- Let's say we have a feature that is always set to 1 regardless of what the input text is.
- This is clearly not an informative feature. However, let's say it was the only one I had...

first, how many weights do I need to learn for this feature?

What is the bias?

- Let's say we have a feature that is always set to 1 regardless of what the input text is.
- This is clearly not an informative feature. However, let's say it was the only one I had...

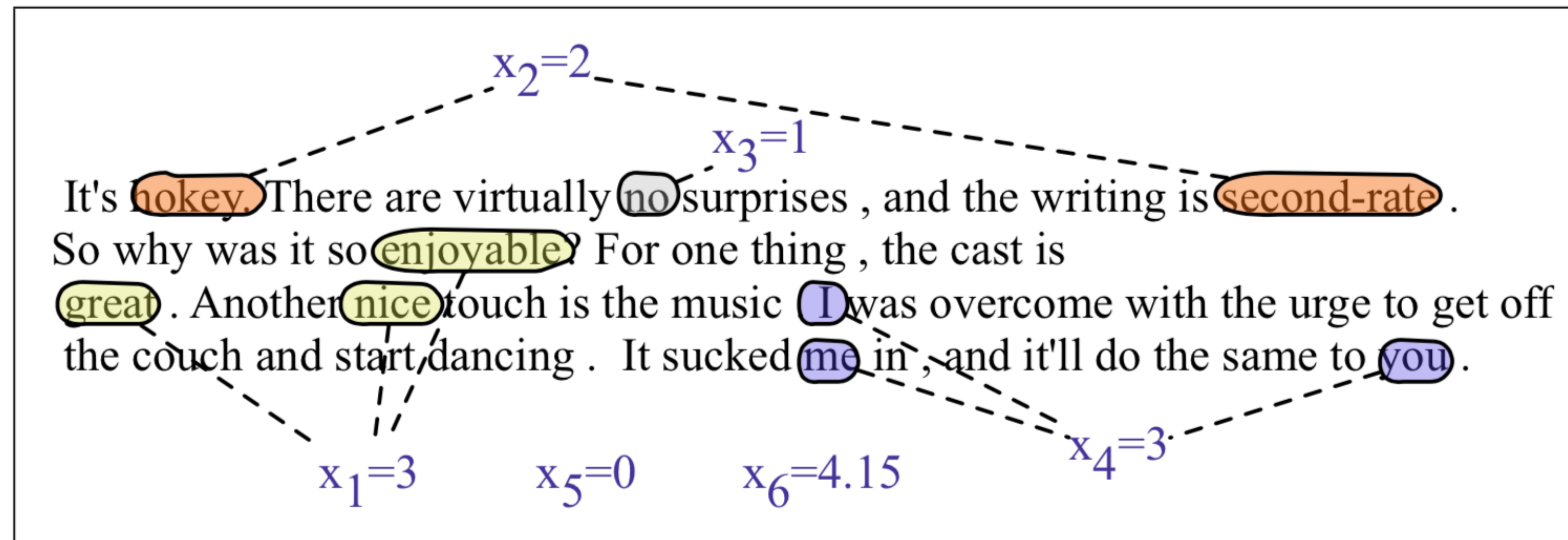
$$\mathbf{w} \cdot \mathbf{x} + \boxed{b}$$

first, how many weights do I need to learn for this feature?

okay... what is the best set of weights for it?

What do we use as features?

Example: Sentiment classification



Var	Definition	Value in Fig. 5.2
x_1	count(positive lexicon) $\in$ doc)	3
x_2	count(negative lexicon) $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	log(word count of doc)	$\ln(64) = 4.15$

Example: Sentiment classification

Var	Definition	Value
x_1	count(positive lexicon) $\in$ doc)	3
x_2	count(negative lexicon) $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	log(word count of doc)	$\ln(64) = 4.15$

- Assume weights $w = [2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$ and bias $b = 0.1$

$$\begin{aligned} p(+|x) &= P(Y = 1|x) = \sigma(w \cdot x + b) \\ &= \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.15] + 0.1) \\ &= \sigma(.805) \\ &= 0.69 \end{aligned}$$

$$\begin{aligned} p(-|x) &= P(Y = 0|x) = 1 - \sigma(w \cdot x + b) \\ &= 0.31 \end{aligned}$$

Feature design

- Most important rule: Data is *key*!
- Linguistic intuition (e.g. part of speech tags, parse trees)
- Complex combinations

$$x_1 = \begin{cases} 1 & \text{if } \textit{Case}(w_i) = \textit{Lower} \\ 0 & \text{otherwise} \end{cases}$$

$$x_2 = \begin{cases} 1 & \text{if } w_i \in \textit{AcronymDict} \\ 0 & \text{otherwise} \end{cases}$$

$$x_3 = \begin{cases} 1 & \text{if } w_i = \textit{St.} \ \& \ \textit{Case}(w_{i-1}) = \textit{Cap} \\ 0 & \text{otherwise} \end{cases}$$

- Feature templates
 - Sparse representations, hash only seen features into index
 - *N*-gram.
 - Advanced: Representation learning (we will see this with neural networks!)

- "Hash only seen features into index": Maintain a vocabulary lookup dictionary `dict[feature_string -> integer_index]` built dynamically:
 - When a feature is encountered during training for the first time, map it to a new unique dimension index (e.g., the 78th unique feature encountered gets index 78).
 - Unseen combinations across the universe never get indexed or allocated in memory.

Multinomial Logistic Regression

Multinomial Logistic Regression

"I need a **permit**." → **permit** is a Noun.

"They will not **permit** this." → **permit** is a Verb.

- What if we have more than 2 classes? (e.g. Part of speech tagging, named entity recognition)

Person (**PER**): Names of individuals (e.g., Ada Lovelace, Alan Turing).

Organization (**ORG**): Companies, institutions, government entities (e.g., Google, Simon Fraser University).

- Need to model $P(y = c|x) \forall c \in \mathcal{C}$
- Generalize **sigmoid** function to **softmax**

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Softmax with $K = 2$
Reduces to Sigmoid

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}} \quad 1 \leq i \leq k$$

Normalization

Softmax

- Similar to sigmoid, softmax squashes values towards 0 or 1
- If $z = [0,1,2,3,4]$, then
 - $\text{softmax}(z) = ([0.0117, 0.0317, 0.0861, 0.2341, 0.6364])$
- For multinomial LR,

$$P(y = c|x) = \frac{e^{w_c \cdot x + b_c}}{\sum_{j=1}^k e^{w_j \cdot x + b_j}}$$

Features in multinomial LR

- Features need to include both **input (x)** and **class (c)**
- Implicit in binary case

Var	Definition	Wt
$f_1(0, x)$	$\begin{cases} 1 & \text{if “!”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	-4.5
$f_1(+, x)$	$\begin{cases} 1 & \text{if “!”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	2.6
$f_1(-, x)$	$\begin{cases} 1 & \text{if “!”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1.3

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

Here, $\mathbf{W}$ is a $K \times d$ weight matrix:

- Row 1 ($\mathbf{w}_0^\top$): Weights for class **Neutral**
- Row 2 ($\mathbf{w}_+^\top$): Weights for class **Positive**
- Row 3 ($\mathbf{w}_-^\top$): Weights for class **Negative**

Same in NN

In binary logistic regression ($y \in \{0, 1\}$), we usually write $z = \mathbf{w}^\top \mathbf{x} + b$ with only one set of weights $\mathbf{w}$.

We could get away with this because a single weight w_i implicitly models the contrast:

- A positive weight pushes towards class 1.
- A negative weight pushes towards class 0.

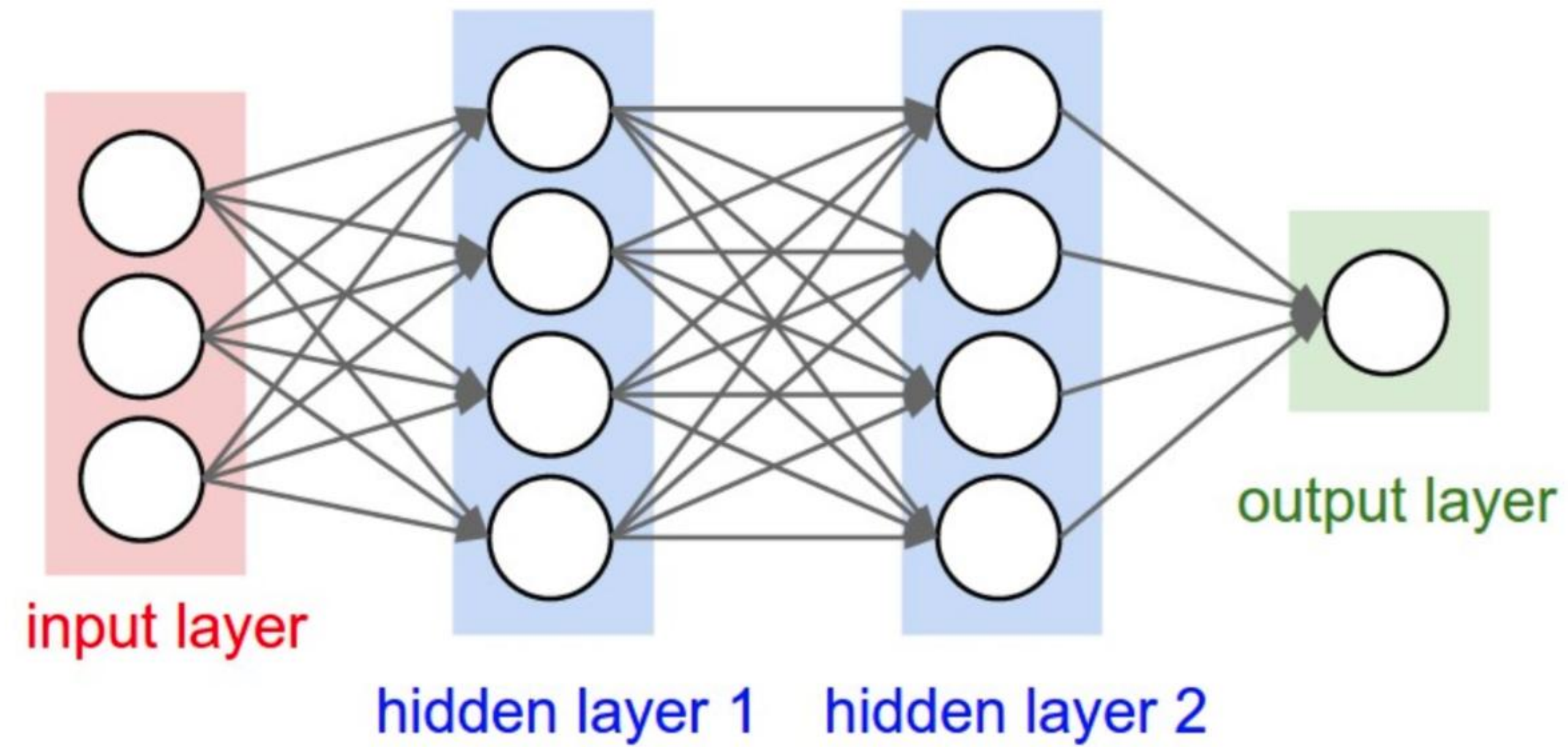
Logistic Regression: what's good and what's not

- More freedom in designing features
- No strong independence assumptions like Naive Bayes
- More robust to correlated features (“San Francisco” vs “Boston”) —LR is likely to work better than NB
- Can even have the same feature twice! (*why?*)
- **However:** not as good on small datasets (compared to Naive Bayes)
- Interpreting learned weights can be challenging

Relationship to neural networks

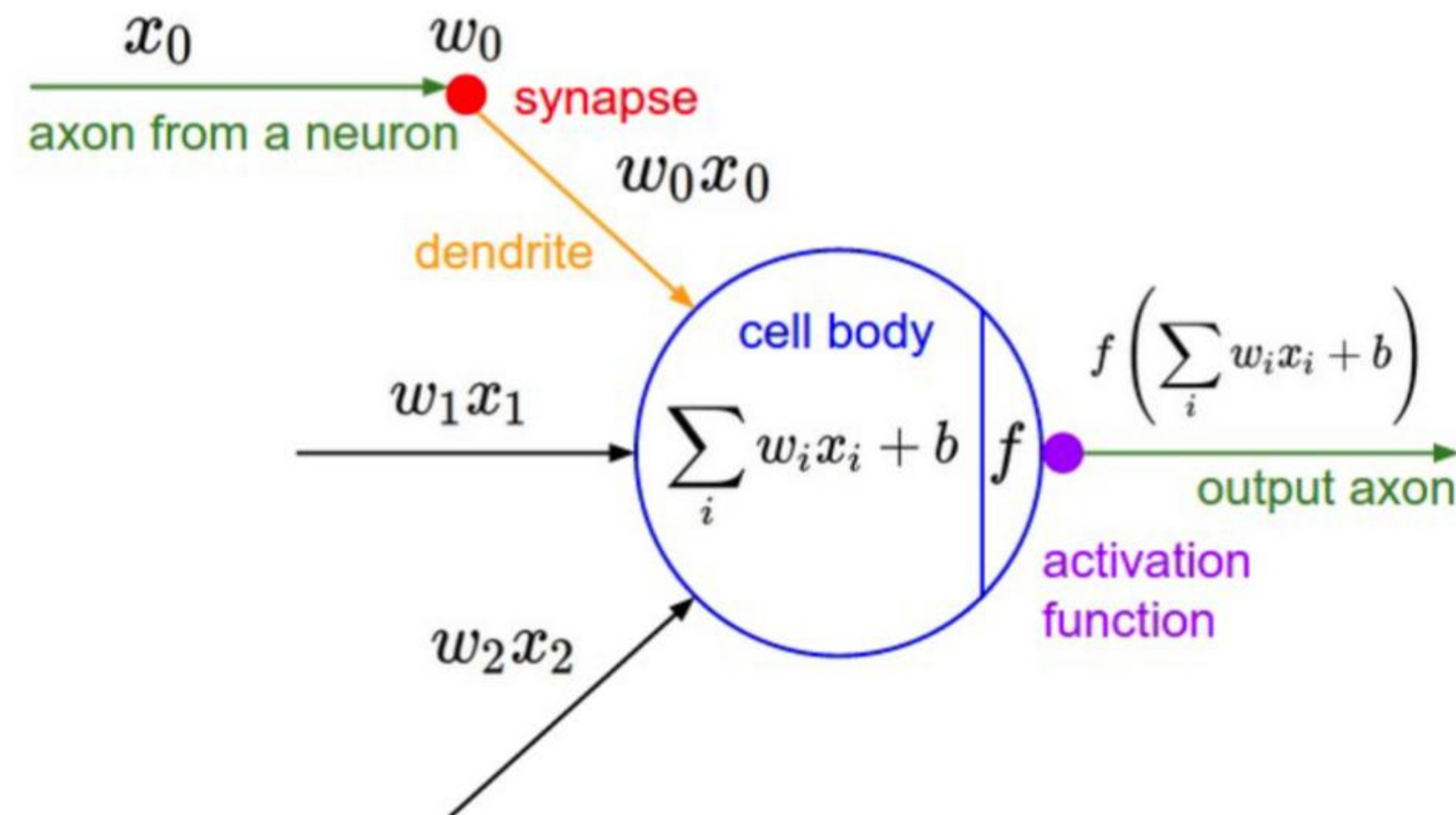
Feed-forward NNs

- Input: $x_1, \dots, x_d$
- Output: $y \in \{0,1\}$



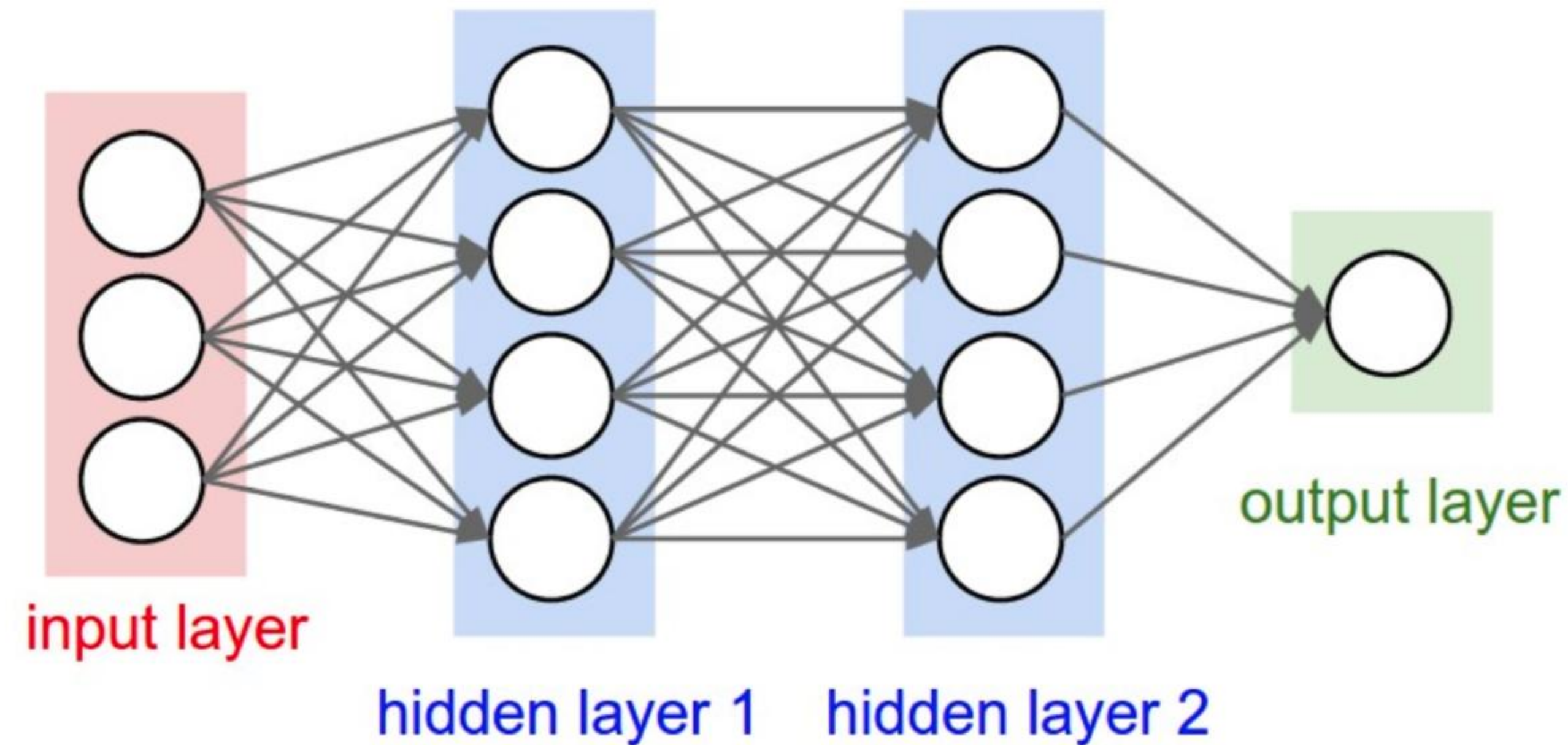
An artificial neuron

- A neuron is a computational unit that has scalar inputs and an output
- Each input has an associated weight.
- The neuron multiplies each input by its weight, sums them, applied a **nonlinear function** to the result, and passes it to its output.



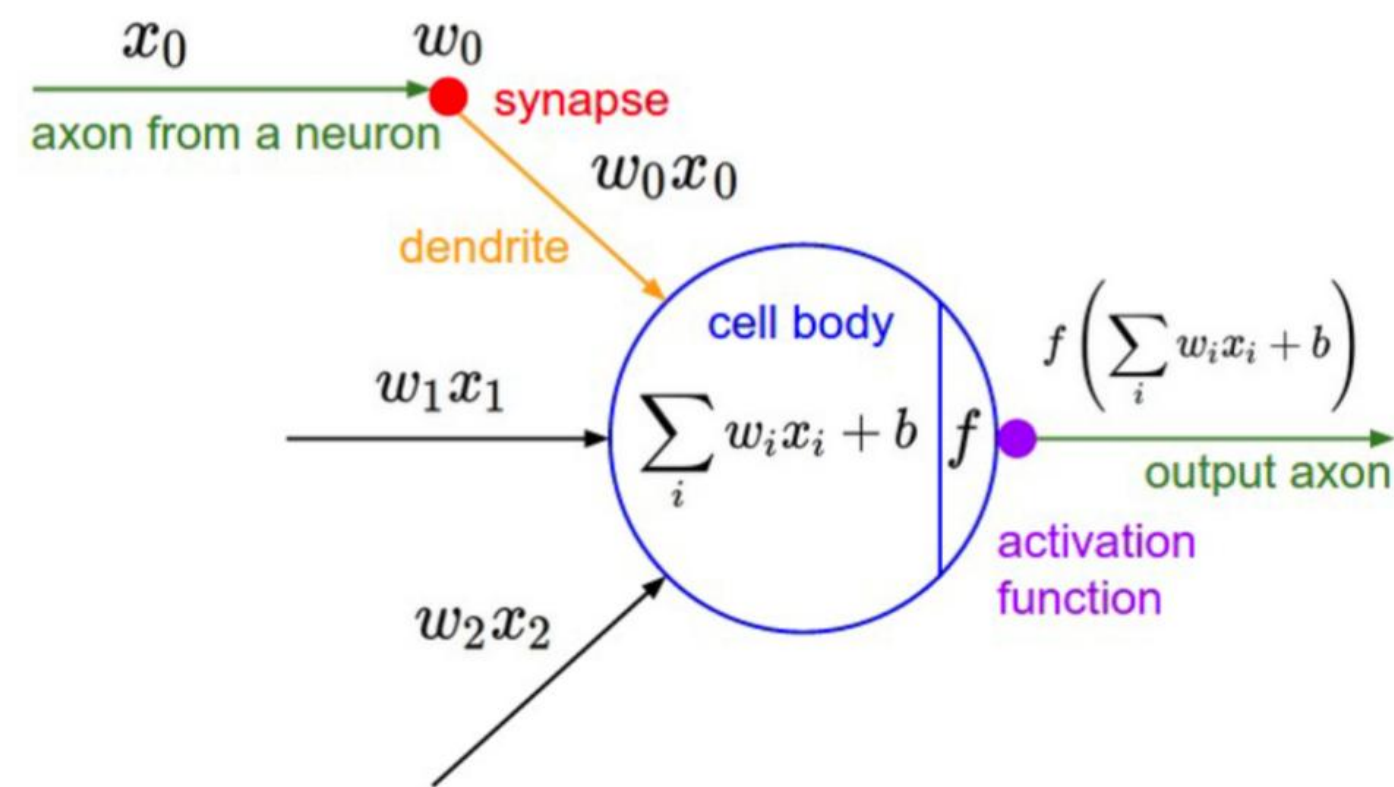
Neural networks

- The neurons are connected to each other, forming a **network**
- The output of a neuron may feed into the inputs of other neurons



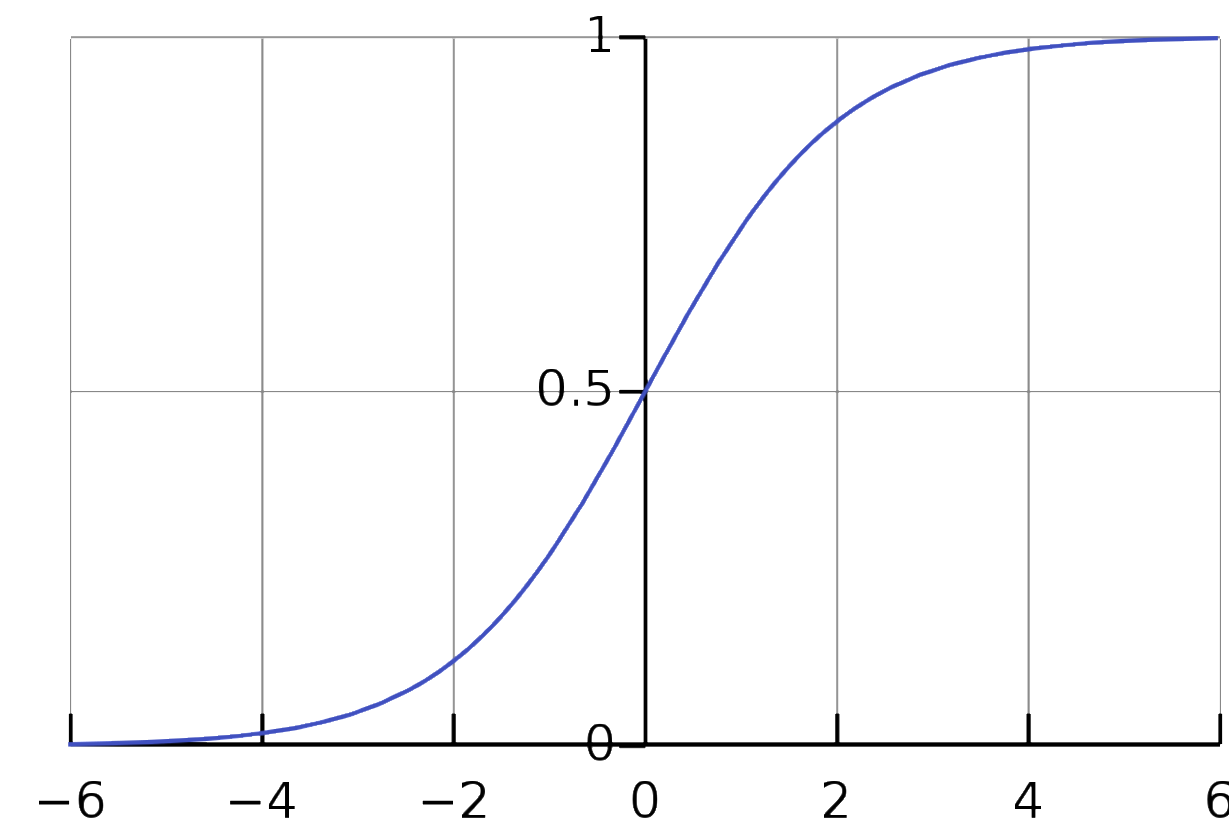
- Feed forward network
- Fully connected network

A neuron can be a binary logistic regression unit

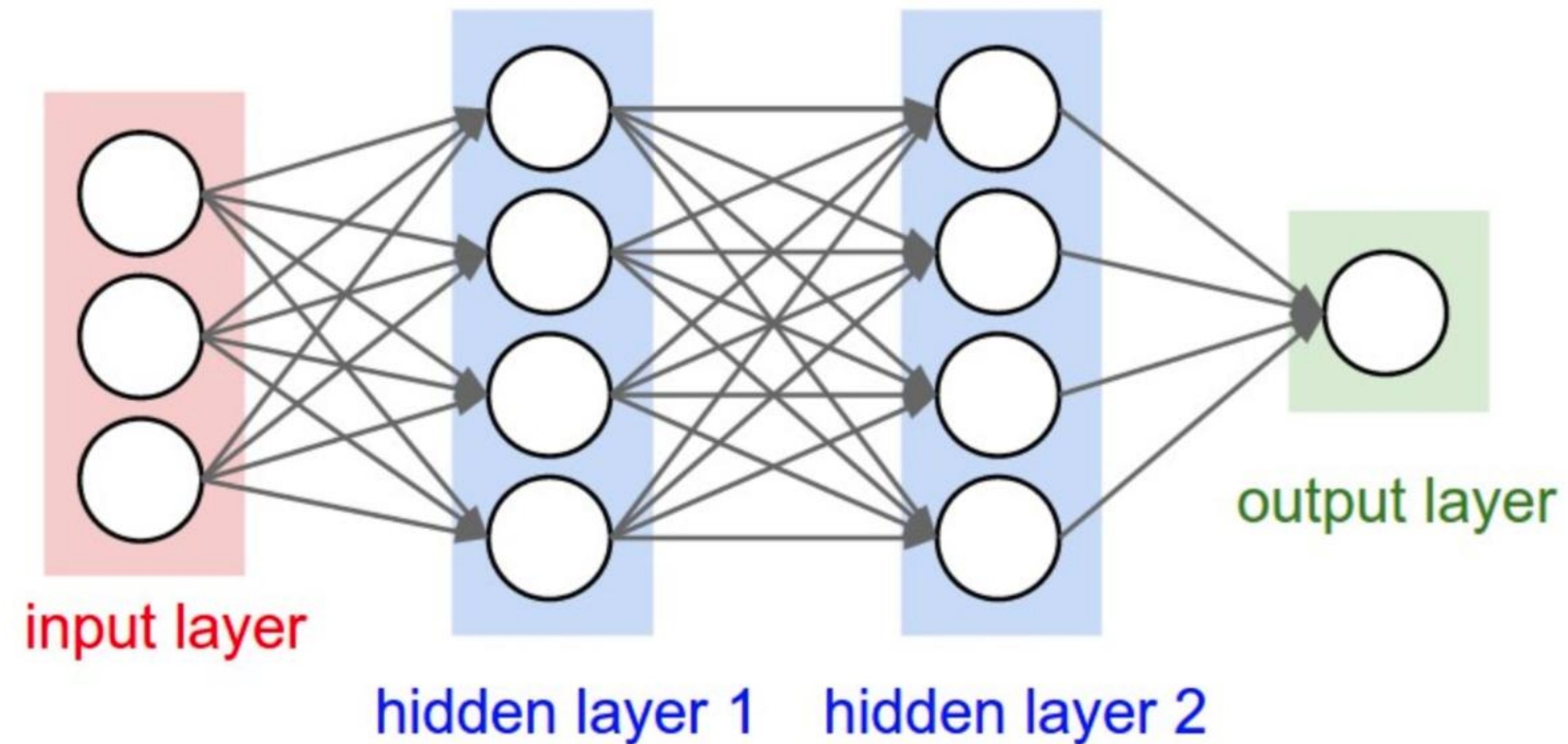


$$f(z) = \frac{1}{1 + e^{-z}}$$

$$h_{\mathbf{w},b}(\mathbf{x}) = f(\mathbf{w}^\top \mathbf{x} + b)$$



A neural network
= running several logistic regressions at the same time

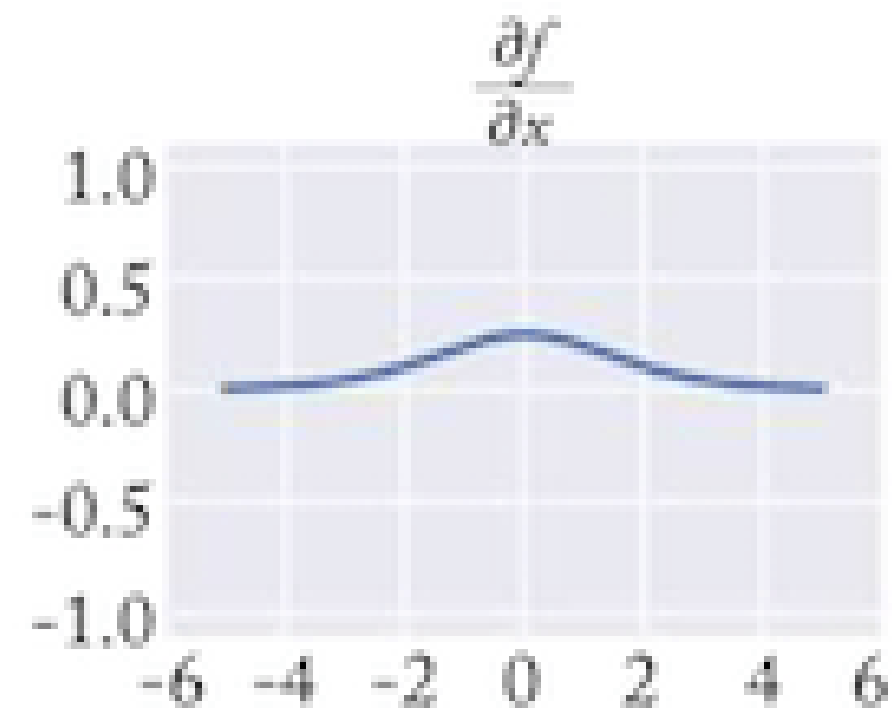
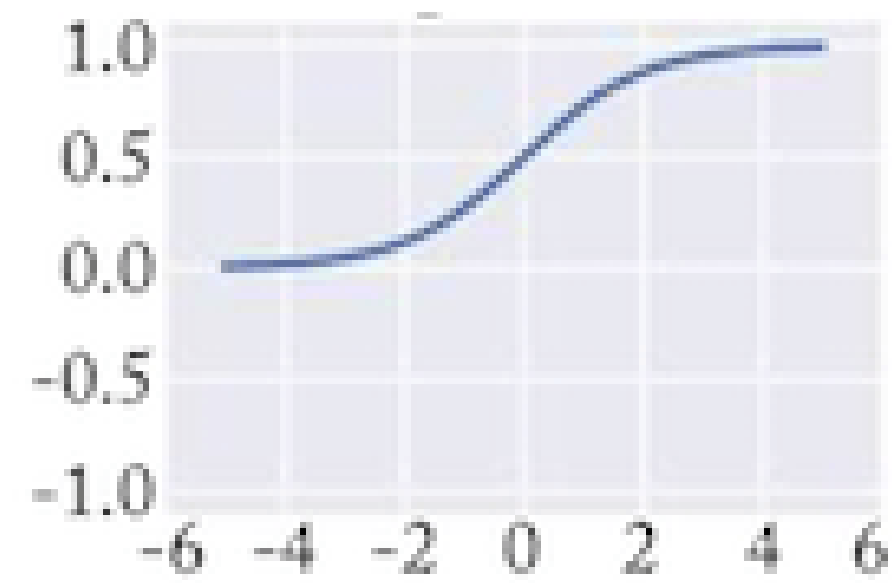


- If we feed a vector of inputs through a bunch of logistic regression functions, then we get a vector of outputs...
- which we can feed into another logistic regression function

NN has other activation functions

sigmoid

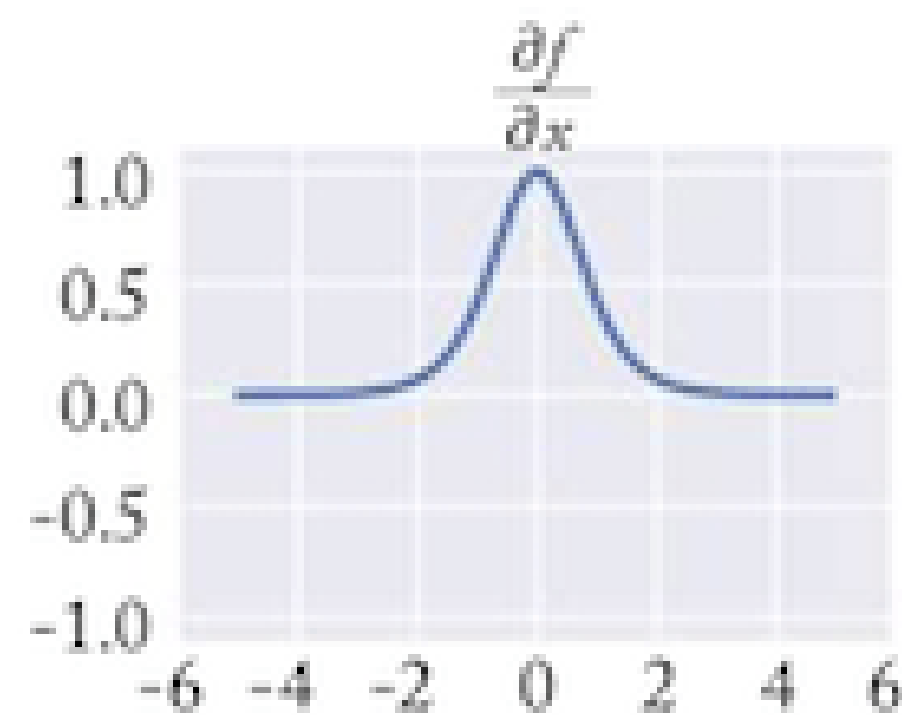
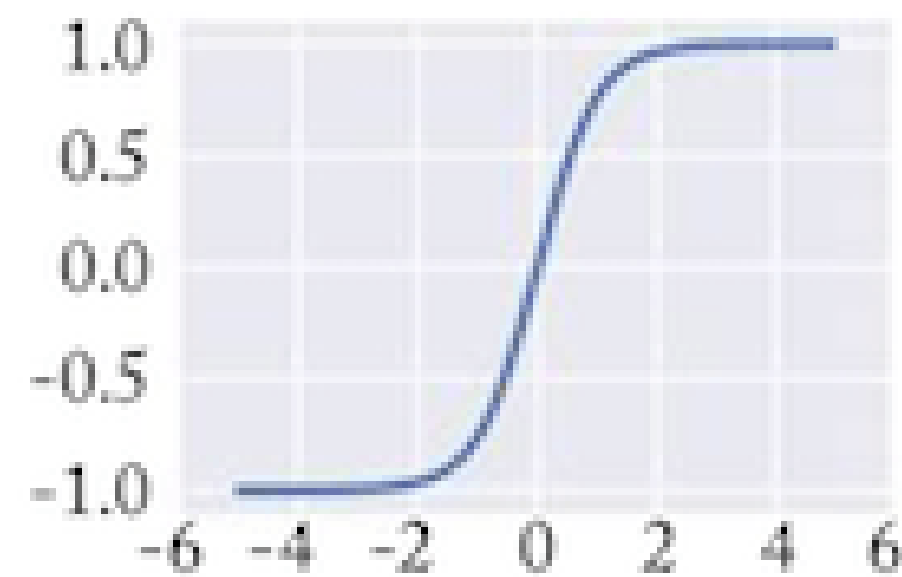
$$f(z) = \frac{1}{1 + e^{-z}}$$



$$f'(z) = f(z) \times (1 - f(z))$$

Zero centered
tanh

$$f(z) = \frac{e^{2z} - 1}{e^{2z} + 1}$$

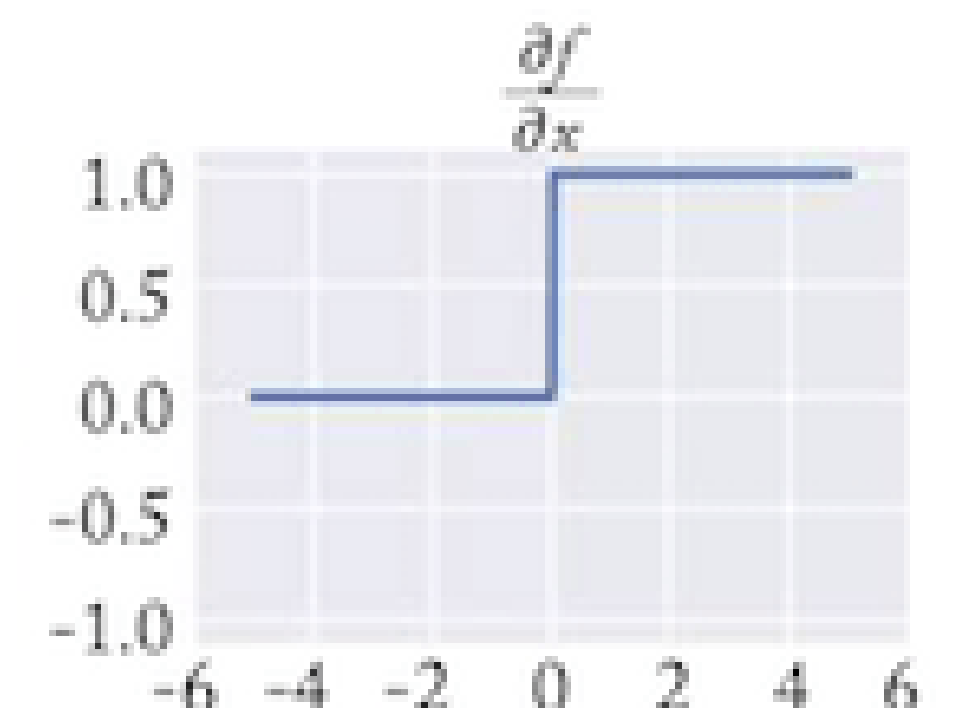
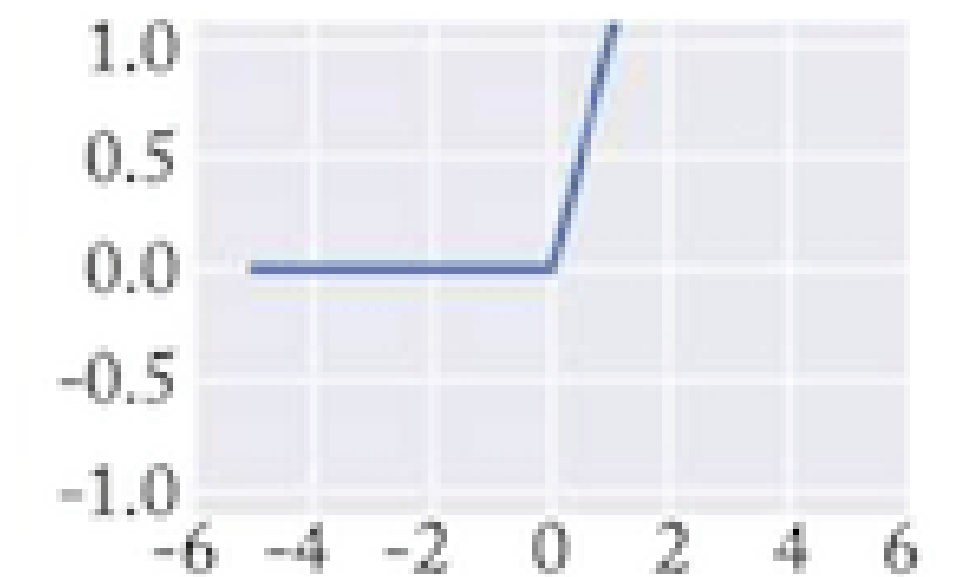


$$f'(z) = 1 - f(z)^2$$

Advantages of ReLU?

ReLU
(rectified linear unit)

$$f(z) = \max(0, z)$$



$$f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

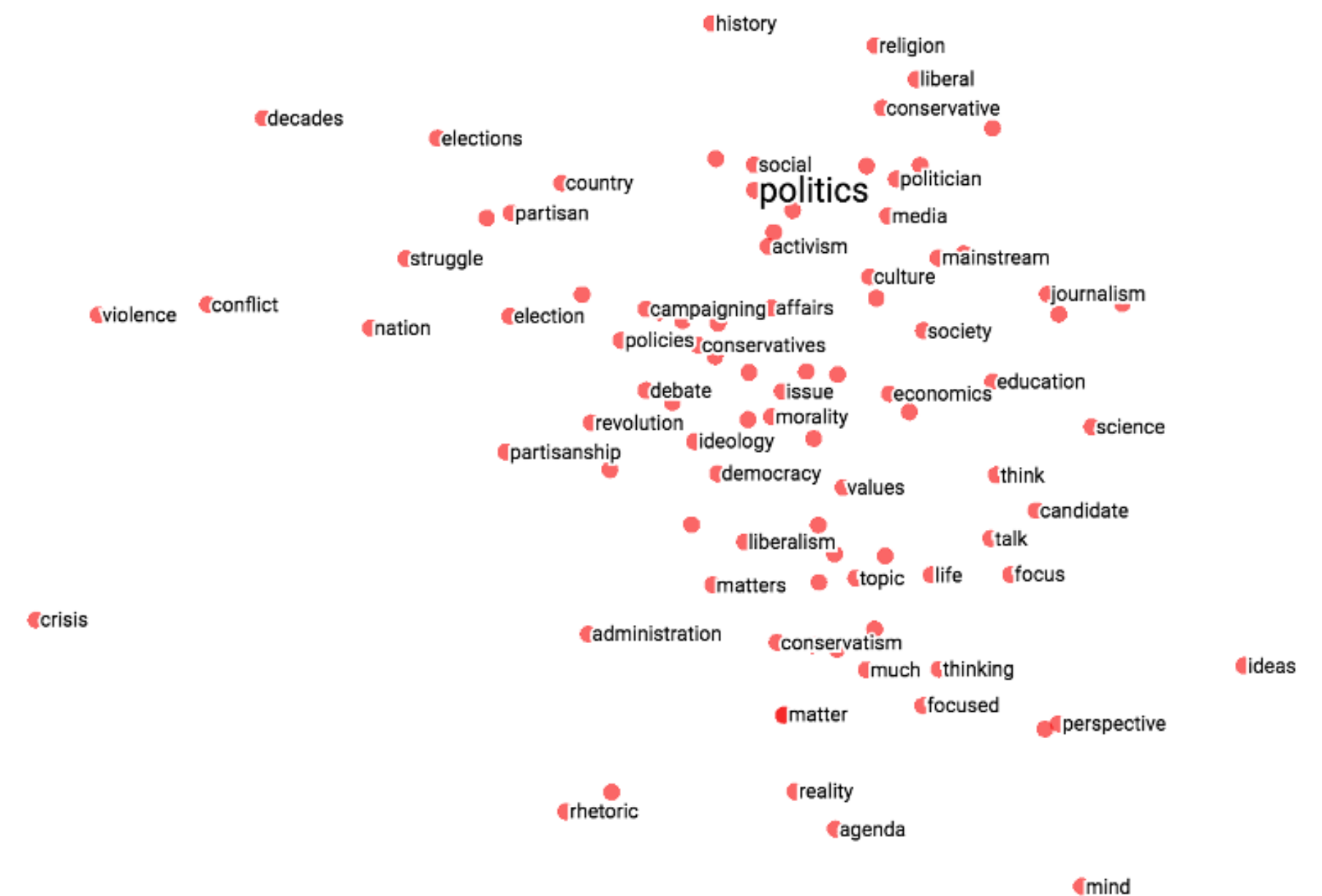
Advantages of neural networks

- Learn feature representations (no longer have to hand craft features)

Hand-crafted features

Var	Definition	Value in Fig. 5.2
x_1	count(positive lexicon) $\in$ doc)	3
x_2	count(negative lexicon) $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	log(word count of doc)	$\ln(64) = 4.15$

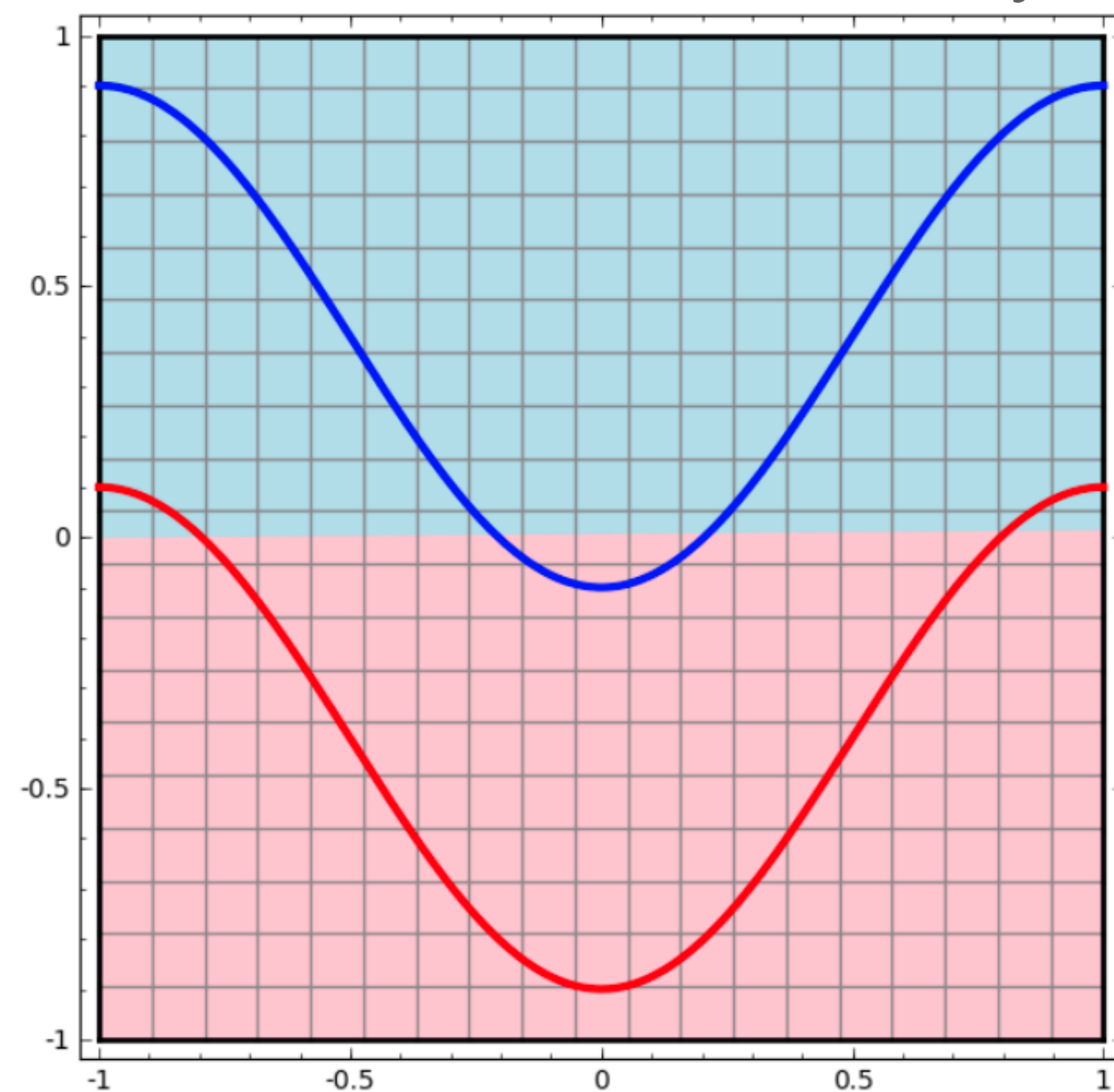
Learned representations



Advantages of neural networks

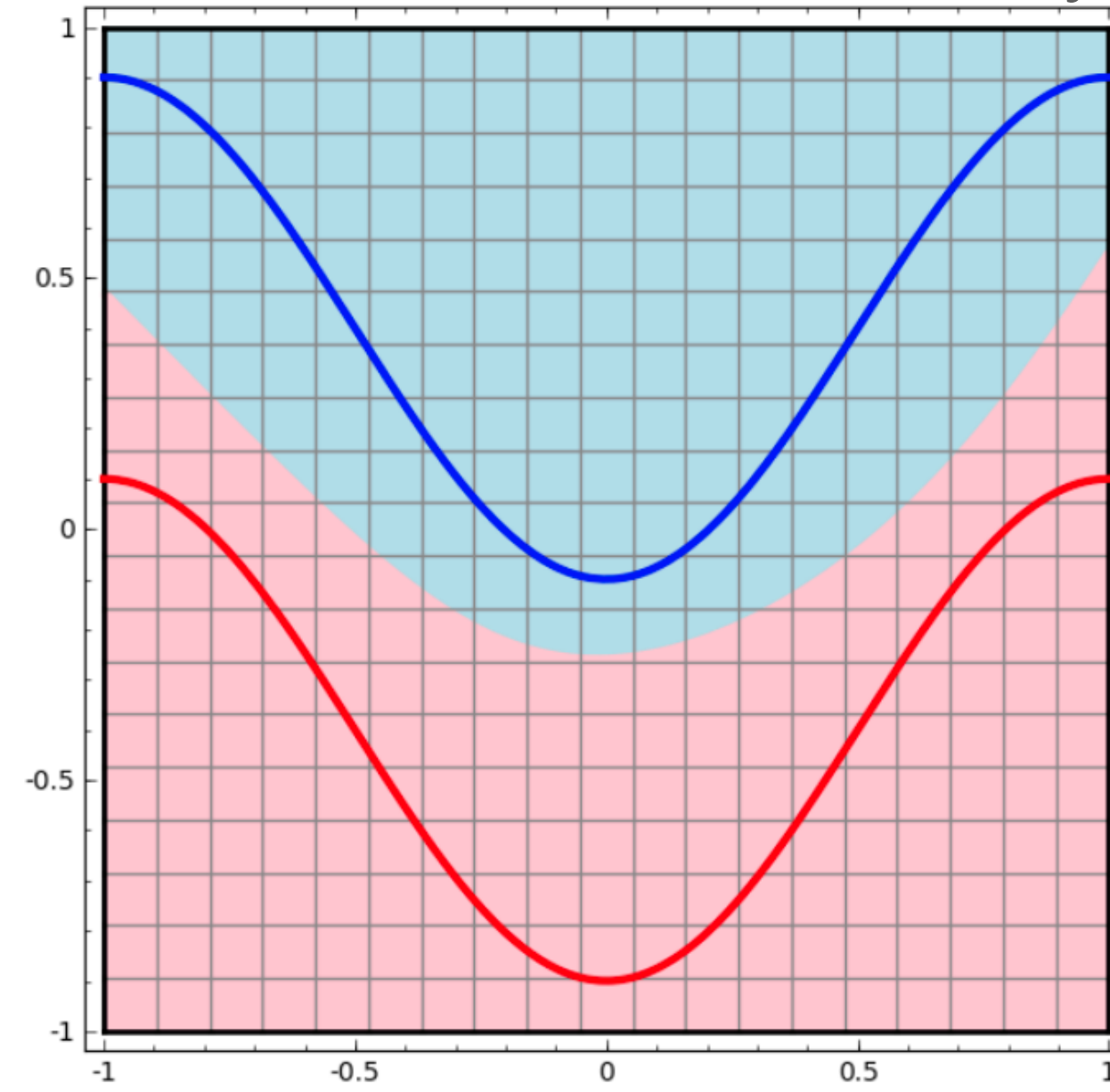
- Learn feature representations (no longer have to hand code features)
- Multiple layers allow for more complex functions with non-linear decision boundaries

Linear decision boundary



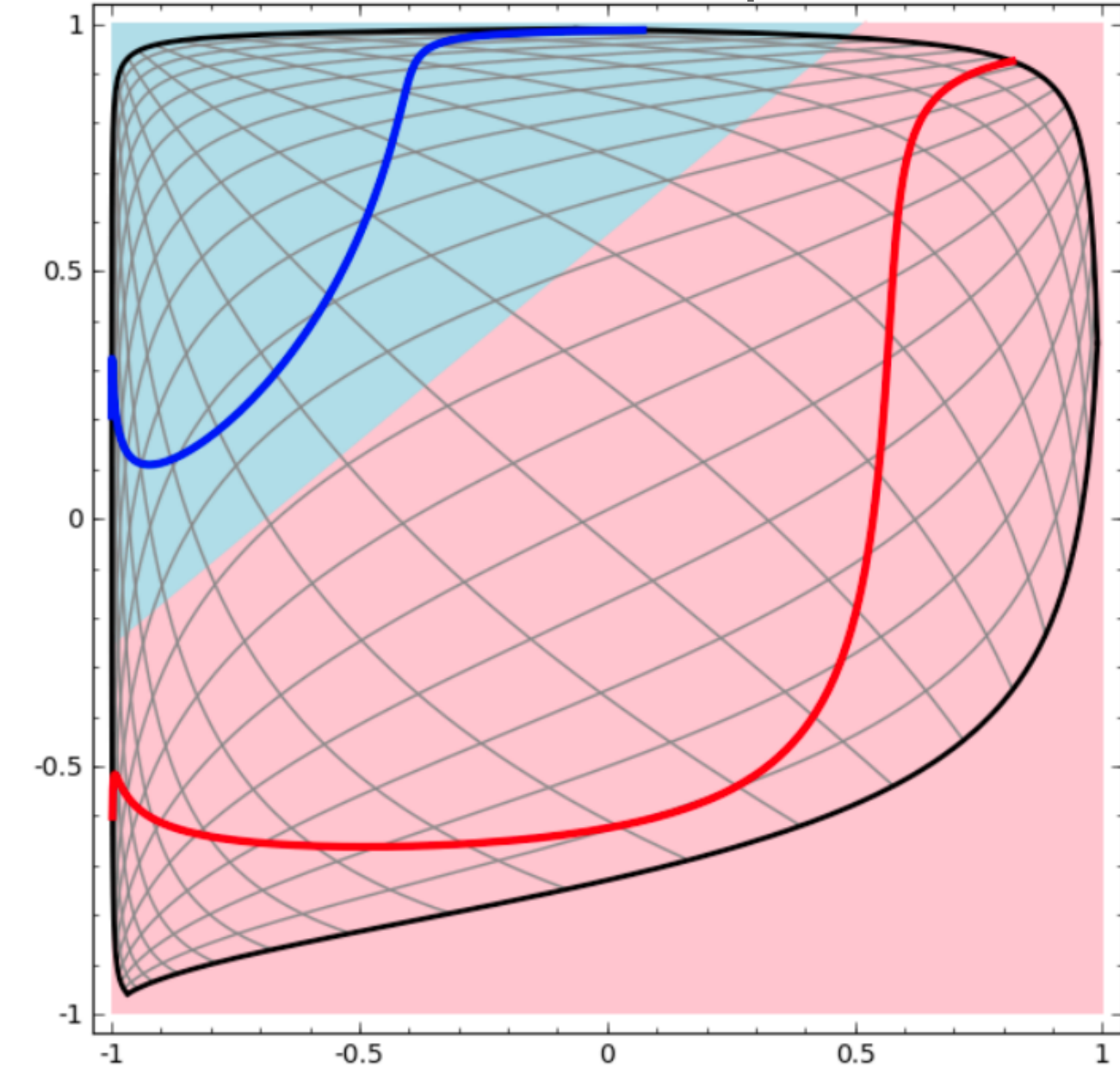
Linear classifier

Non-linear decision boundary



Neural Networks

Linear decision boundary
in transformed space



Hidden representations
are linearly separable!

Advantages of neural networks

- Learn feature representations (no longer have to hand code features)
- Multiple layers allow for more complex functions with non-linear decision boundaries
- Can build up complex models by connecting “neurons” into building blocks, and by connecting these building blocks together
- Optimization is mostly taken care of by auto-differentiation libraries
- Development of flexible deep learning libraries allow researchers and developers to focus more on modelling the problem

Back to Logistic Regression

How to learn the parameters?

Learning

- We have our **classification function** - how to assign weights and bias?
- **Goal:** predicted label $\hat{y}$ as close as possible to actual label y
- **Distance metric/Loss function** between $\hat{y}$ and y :
$$L(\hat{y}, y)$$
- **Optimization algorithm** for updating weights

Loss function

- Assume $\hat{y} = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$
- $L(\hat{y}, y) = \text{Measure of difference between } \hat{y} \text{ and } y$. But what form?
- Maximum likelihood estimation (conditional):
 - Choose $\mathbf{w}$ and b such that $\log P(y|x)$ is maximized for true labels y paired with input x
 - Similar to language models!
 - $\max \log P(w_t | w_{t-n}, \dots, w_{t-1})$ given a corpus

Component	Logistic Regression	Language Model (LM)
Context / Input (x)	Feature vector x (e.g., words in an email)	Previous words context $w_{t-n}, \dots, w_{t-1}$
Target / Label (y)	Class label $y \in \{0, 1\}$ (or $c \in \{1, \dots, K\}$)	Next word $w_t \in V$ (from vocabulary V)
Model Output	$P(y x) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$	$P(w_t w_{t-n}, \dots, w_{t-1}) = \text{softmax}(\mathbf{W}\mathbf{h} + \mathbf{b})$
Objective (MLE)	$\max_{\mathbf{w}, b} \sum \log P(y x)$	$\max_{\theta} \sum \log P(w_t w_{t-n}, \dots, w_{t-1})$

Cross Entropy loss

- Assume a single data point (x, y) and two classes (Binary Cross-Entropy Loss)

$y \in \{0, 1\}$ be the true ground-truth label.

$\hat{y} = P(y = 1 | x)$ be the model's predicted probability that the label is 1.

$1 - \hat{y} = P(y = 0 | x)$ be the predicted probability that the label is 0.

If true $y = 1$: $P(y | x) = \hat{y}^1(1 - \hat{y})^0 = \hat{y}$

If true $y = 0$: $P(y | x) = \hat{y}^0(1 - \hat{y})^1 = 1 - \hat{y}$

- Classifier probability: $P(y|x) = \hat{y}^y(1 - \hat{y})^{1-y}$

Recall in L1b P43, cross entropy :

p: true prob distribution

q: model distribution

- Log probability: $\log P(y|x) = \log[\hat{y}^y(1 - \hat{y})^{1-y}]$

Correction: In machine learning $\log = \ln$

$$= y \log \hat{y} + (1 - y) \log(1 - \hat{y})$$

$$H(p, q) = - \sum_x p(x) \log q(x) = \mathbb{E}_{x \sim p}[-\log q(x)]$$

Because: maximize likelihood = minimize error (loss). Negative log-likelihood

- Loss: $-\log P(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$

$$y = 1 \Rightarrow -\log \hat{y}$$

$$y = 0 \Rightarrow -\log(1 - \hat{y})$$

Example: Computing CE Loss

Var	Definition	Val
x_1	count(positive lexicon) $\in$ doc)	3
x_2	count(negative lexicon) $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	log(word count of doc)	$\ln(64) = 4.15$

- Assume weights $w = [2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$ and bias $b = 0.1$
- If $y = 1$ (positive sentiment), $L_{CE} = -\log(0.69) = 0.37$
- If $y = 0$ (negative sentiment), $L_{CE} = -\log(0.31) = 1.17$

Cross Entropy loss

- Assume n data points $(x^{(i)}, y^{(i)})$
- Classifier probability: $\prod_{i=1}^n P(y^{(i)} | x^{(i)}) = \prod_{i=1}^n \hat{y}^y (1 - \hat{y})^{1-y}$
- Loss: $-\log \prod_{i=1}^n P(y^{(i)} | x^{(i)}) = -\sum_{i=1}^n \log P(y^{(i)} | x^{(i)})$

$$L_{CE} = -\sum_{i=1}^n [y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

Compare to P37: Loss: $-\log P(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$

Maximizing likelihood = Minimizing cross entropy

Properties of CE Loss

- $L_{CE} = - \sum_{i=1}^n [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)})]$
 $L_{CE}(y^{(i)}, x^{(i)}; \theta)$

Recall in L1b P43, cross entropy :

p: true prob distribution

q: model distribution

$$H(p, q) = - \sum_x p(x) \log q(x) = \mathbb{E}_{x \sim p} [-\log q(x)]$$

- Ranges from 0 (perfect predictions) to ∞

- Lower the value, better the classifier

- **Cross-entropy** between the true distribution $P(y|x)$ and predicted distribution $P(\hat{y}|x)$

$$L_{CE} = - \frac{n_{y=1}}{n} \log \hat{y} - \frac{n_{y=0}}{n} \log (1 - \hat{y}) = - \sum_{y=0}^1 p(y|x) \log p(\hat{y}|x)$$

$q(x)$

Optimization

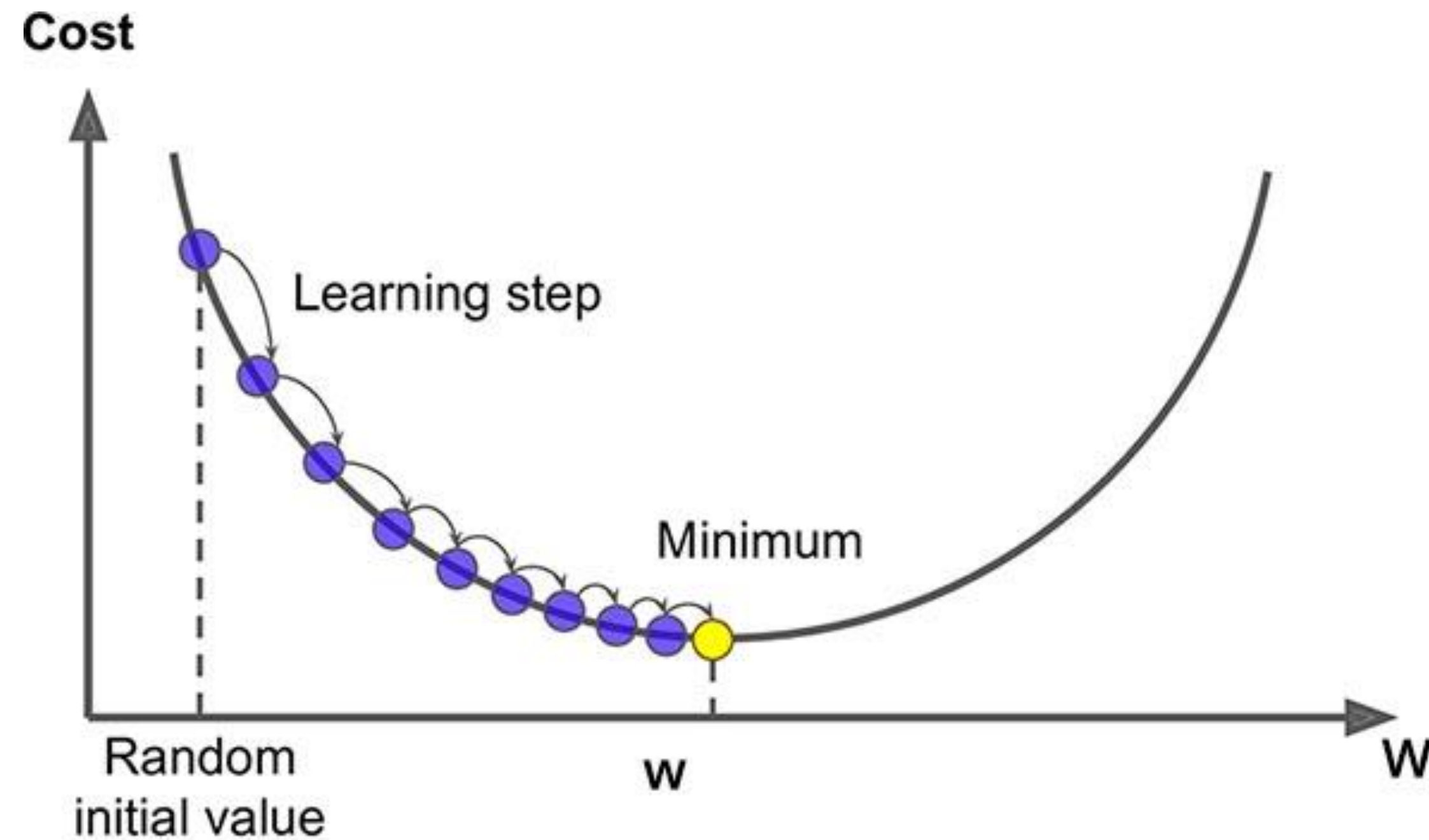
- We have our **classification function** and **loss function** - how do we find the best w and b ?

$$\theta = [w; b] \quad \text{Two scalars}$$

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n L_{CE}(y^{(i)}, x^{(i)}; \theta)$$

- **No close form solution:** need to numerically determine optimal solution
- Gradient descent:
 - Find direction of steepest slope
 - Move in the opposite direction

Gradient descent (1-D)



learning rate
or step size

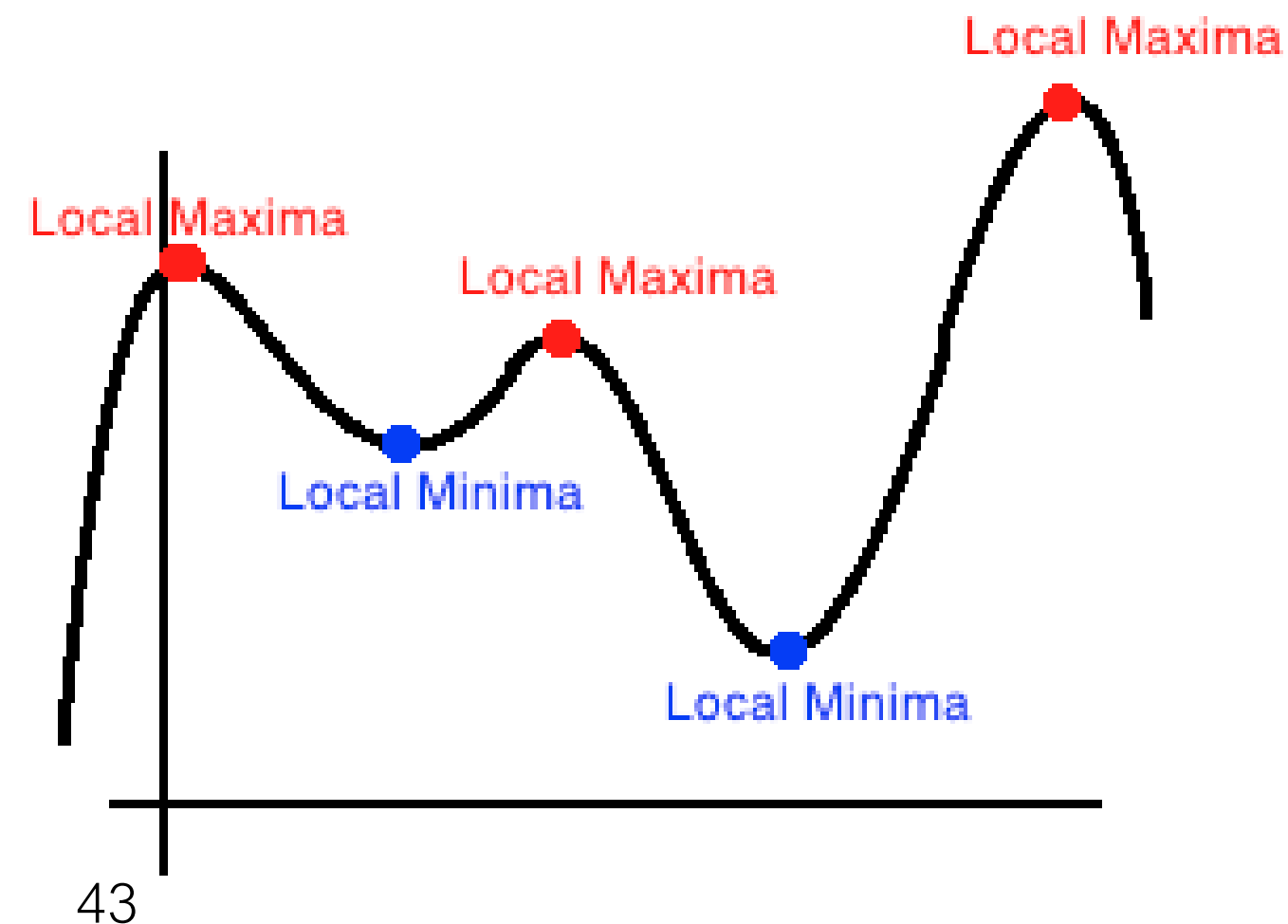
$$\theta^{t+1} = \theta^t - \eta \frac{d}{d\theta} L(f(x; \theta), y)$$

$$L_{CE}(y^{(i)}, x^{(i)}; \theta)$$

Loss function we are optimizing

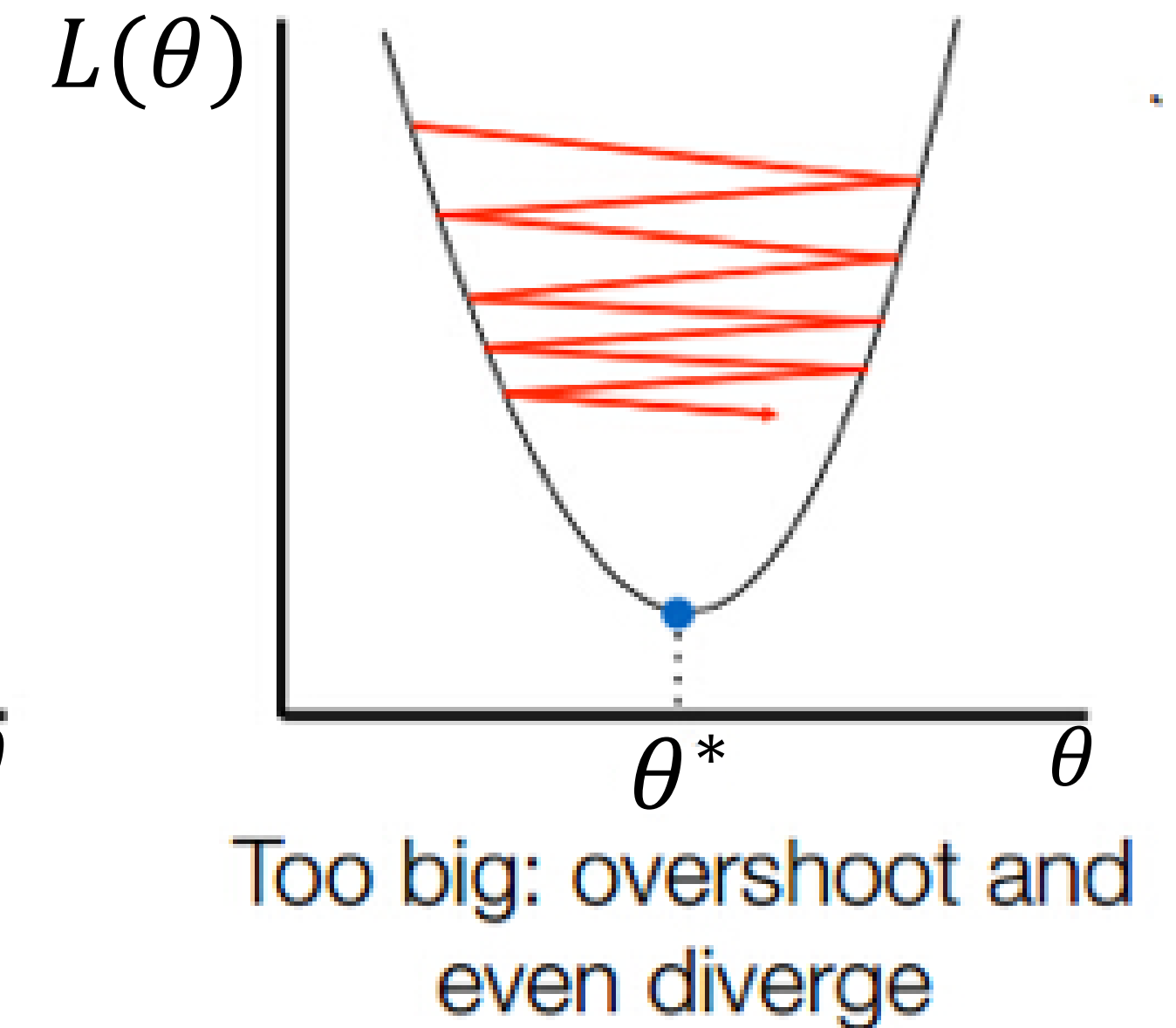
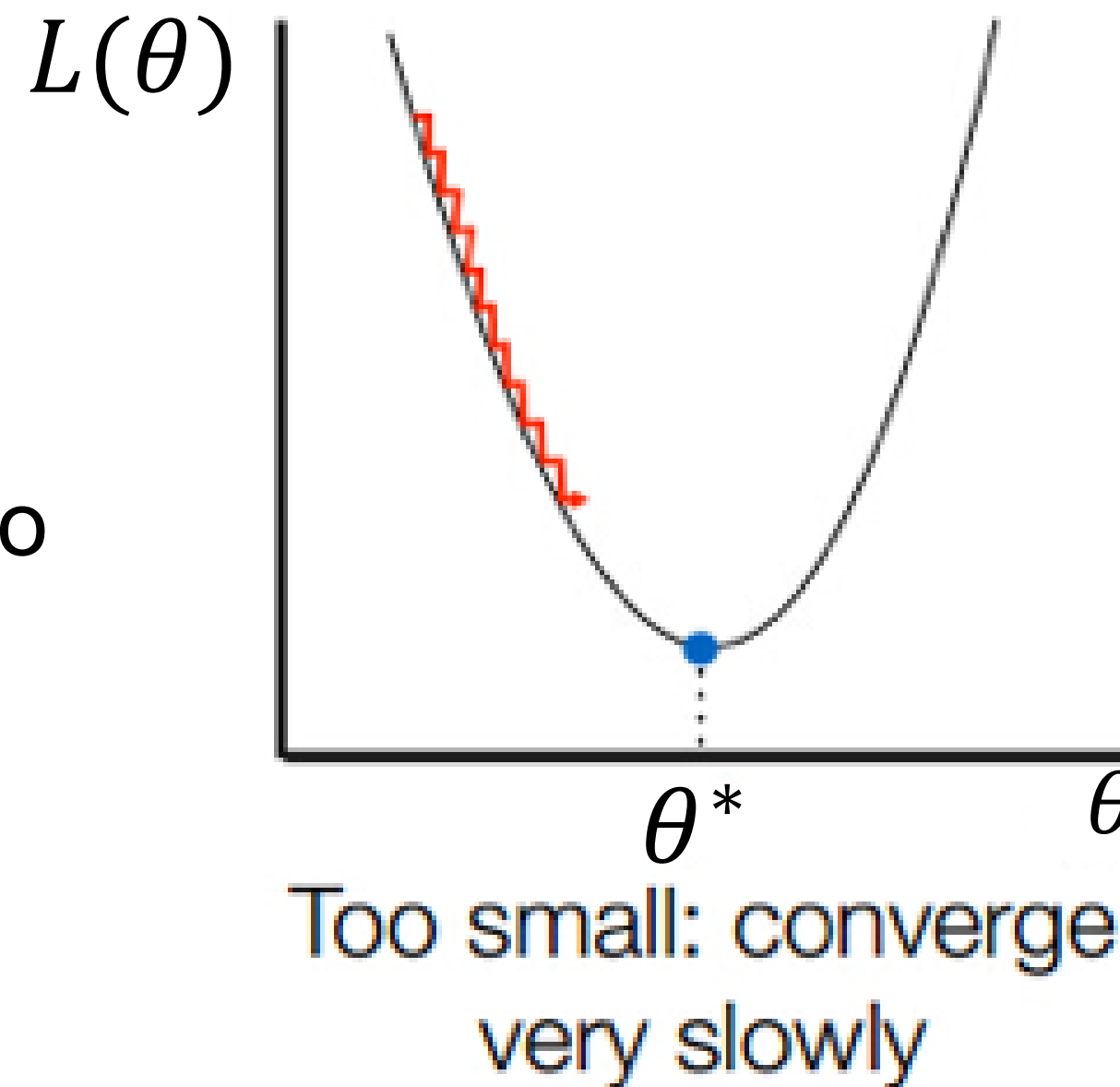
Gradient descent for LR

- Cross entropy loss for logistic regression is **convex** (i.e. has only one global minimum)
- No local minima to get stuck in
- Deep neural networks are not so easy
- Non-convex



Learning Rate

- Updates: $\theta^{t+1} = \theta^t - \eta \frac{d}{d\theta} L(f(x; \theta), y)$
- Magnitude of movement along gradient
- Higher/faster learning rate = larger updates to parameters



Gradient descent with vector weights

- In LR: weight w is a vector
- Express slope as a partial derivative of loss w.r.t each weight:

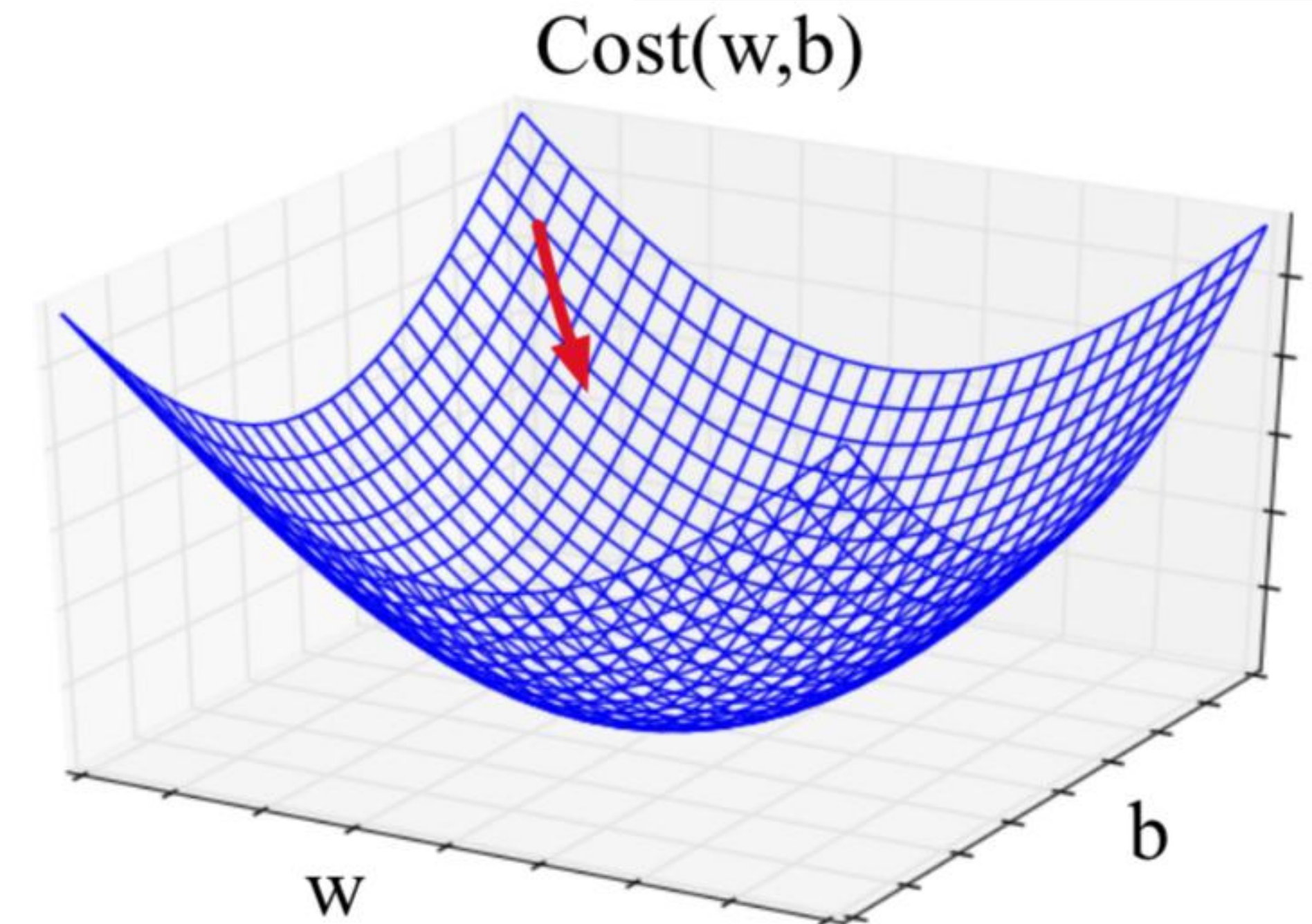
$\nabla_{\theta} L(f(x; \theta), y) = \begin{bmatrix} \frac{\partial}{\partial w_1} L(f(x; \theta), y) \\ \frac{\partial}{\partial w_2} L(f(x; \theta), y) \\ \vdots \\ \frac{\partial}{\partial w_n} L(f(x; \theta), y) \end{bmatrix}$

∇ Nabla
 del theta of L
 del L with respect to theta
 gradient of L with respect to theta

- Updates: $\theta^{(t+1)} = \theta^t - \eta \nabla L(f(x; \theta), y)$

Cost function

$$J(\mathbf{w}, b) \quad \text{Cost}(\mathbf{w}, b) = \frac{1}{N} \sum_{i=1}^N L(f(x^{(i)}; \mathbf{w}, b), y^{(i)})$$



$$\theta^{t+1} = \theta^t - \eta \frac{d}{d\theta} L(f(x; \theta), y)$$

From two scalars to a vector

Gradient for logistic regression

From P40

$$\bullet L_{CE} = - \sum_{i=1}^n [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

$L_{CE}(y^{(i)}, x^{(i)}; \theta)$

L_{CE}

$$= - \sum_{i=1}^n [y^{(i)} \log \sigma(w \cdot x^{(i)} + b) + (1 - y^{(i)}) \log(1 - \sigma(w \cdot x^{(i)} + b))]$$

This is one of the previous conceptual homework

Gradient, $\frac{dL_{CE}(w, b)}{dw_j} = \sum_{i=1}^n [\sigma(w \cdot x^{(i)} + b) - y^{(i)}] x_j^{(i)}$

Hint:

$$\frac{\partial L}{\partial z} = \hat{y} - y = \sigma(z) - y$$

$$\frac{dL_{CE}(w, b)}{db} = \sum_{i=1}^n [\sigma(w \cdot x^{(i)} + b) - y^{(i)}]$$

Will explain how next.

Properties of the logistic function

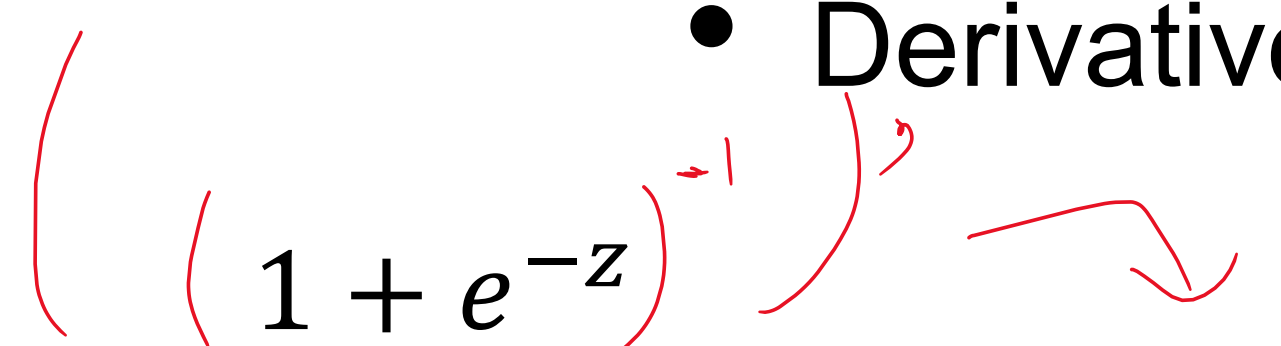
- Relationship to $\sigma(-z)$

$$\sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{e^z+1} = \frac{1+e^z-1}{1+e^z} = 1 - \sigma(-z)$$

$$\Rightarrow \sigma(-z) = 1 - \sigma(z)$$

- Derivative

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$


$$\frac{d\sigma(z)}{dz} = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \frac{1}{e^z+1} = \sigma(z)\sigma(-z) = \sigma(z)(1 - \sigma(z))$$

Taking the gradient

$$L_{CE} = - \sum_{i=1}^n [y^{(i)} \log \sigma(w \cdot x^{(i)} + b) + (1 - y^{(i)}) \log(1 - \sigma(w \cdot x^{(i)} + b))]$$

Consider cross-entropy loss for the i th instance / data point with $z = w \cdot x^{(i)} + b$

$$\frac{dL_{CE}^{(i)}}{dw_j} = \boxed{\frac{dL_{CE}^{(i)}}{dz}} \frac{dz}{dw_j} \qquad \frac{dL_{CE}^{(i)}}{db} = \boxed{\frac{dL_{CE}^{(i)}}{dz}} \frac{dz}{db}$$

$$\frac{dL_{CE}^{(i)}}{dz} = - \left[y^{(i)} \frac{\sigma'(z)}{\sigma(z)} + (1 - y^{(i)}) \frac{-\sigma'(z)}{1 - \sigma(z)} \right] \quad \leftarrow \frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

$$= - \left[y^{(i)} \frac{\sigma(z)(1 - \sigma(z))}{\sigma(z)} + (1 - y^{(i)}) \frac{-\sigma(z)(1 - \sigma(z))}{1 - \sigma(z)} \right]$$

$$= -[y^{(i)}(1 - \sigma(z)) + (1 - y^{(i)})(-\sigma(z))] = -[y^{(i)} - \sigma(z)] = \sigma(z) - y^{(i)}$$

$$\boxed{\frac{dL_{CE}^{(i)}}{dz} = \sigma(z) - y^{(i)}}$$

Taking the gradient

$$L_{CE}$$

$$= - \sum_{i=1}^n [y^{(i)} \log \sigma(w \cdot x^{(i)} + b) + (1 - y^{(i)}) \log(1 - \sigma(w \cdot x^{(i)} + b))]$$
$$z = w \cdot x^{(i)} + b$$

$$\frac{dL_{CE}^{(i)}}{dz} = \sigma(z) - y^{(i)}$$

$$\frac{dL_{CE}^{(i)}}{dw_j} = \frac{dL_{CE}^{(i)}}{dz} \frac{dz}{dw_j} = (\sigma(w \cdot x^{(i)} + b) - y^{(i)}) x_j^{(i)}$$

$$\frac{dz}{dw_j} = x_j^{(i)}$$

$$\frac{dL_{CE}^{(i)}}{db} = \frac{dL_{CE}^{(i)}}{dz} \frac{dz}{db} = (\sigma(w \cdot x^{(i)} + b) - y^{(i)})$$

$$\frac{dz}{db} = 1$$

Stochastic Gradient Descent

- Online optimization
- Compute loss and minimize after each training example

function STOCHASTIC GRADIENT DESCENT($L()$, $f()$, x , y) **returns** θ

where: L is the loss function

f is a function parameterized by θ

x is the set of training inputs $x^{(1)}, x^{(2)}, \dots, x^{(n)}$

y is the set of training outputs (labels) $y^{(1)}, y^{(2)}, \dots, y^{(n)}$

$\theta \leftarrow 0$

repeat til done # see caption

For each training tuple $(x^{(i)}, y^{(i)})$ (in random order)

1. Optional (for reporting): # How are we doing on this tuple?

Compute $\hat{y}^{(i)} = f(x^{(i)}; \theta)$ # What is our estimated output $\hat{y}$?

Compute the loss $L(\hat{y}^{(i)}, y^{(i)})$ # How far off is $\hat{y}^{(i)}$ from the true output $y^{(i)}$?

2. $g \leftarrow \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$ # How should we move θ to maximize loss?

3. $\theta \leftarrow \theta - \eta g$ # Go the other way instead

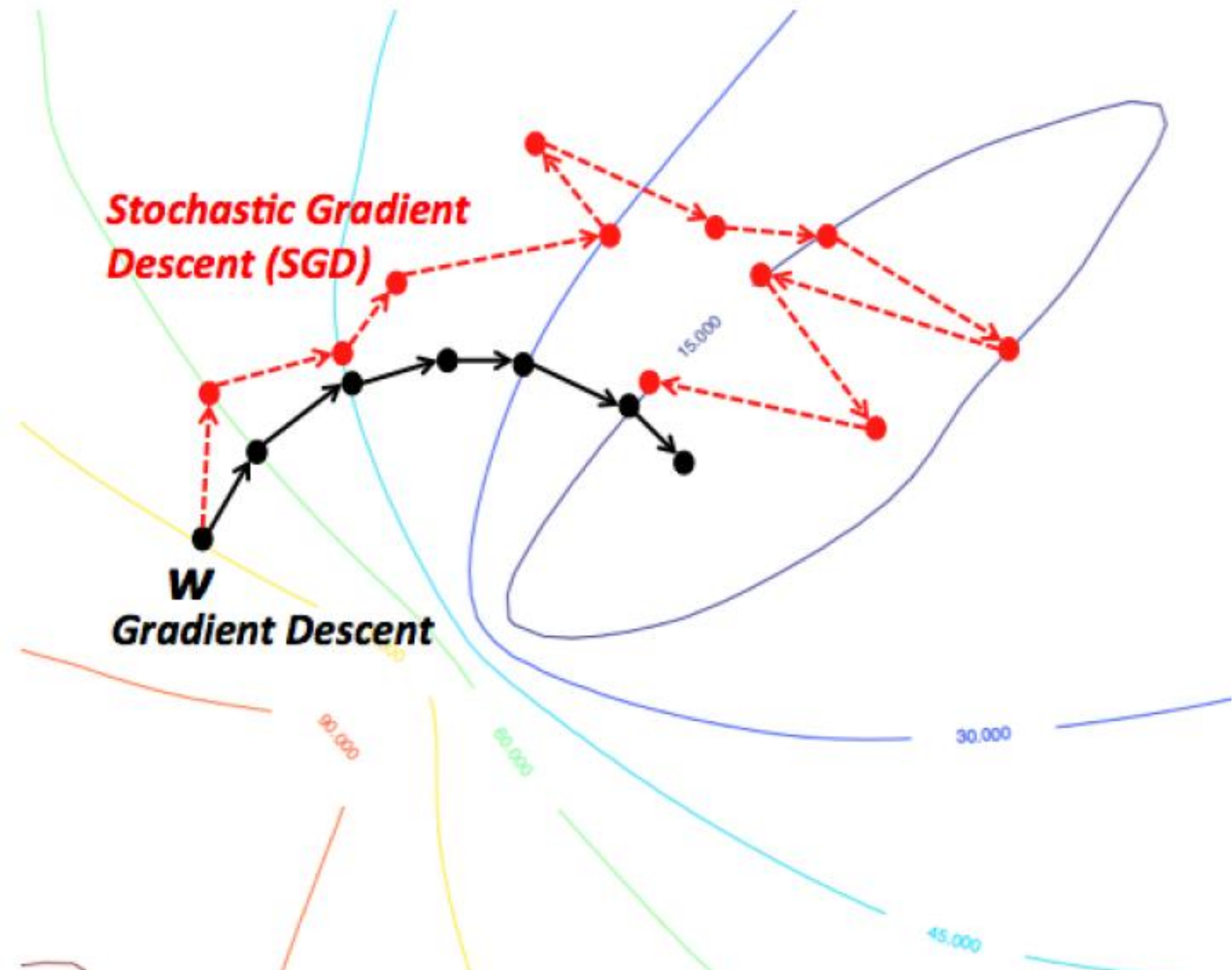
return θ

*Per per
Instance
Loss*

Instead of updating it using a batch of samples

Stochastic Gradient Descent

- Online optimization
- Compute loss and minimize after each training example



Regularization

- Training objective: $\hat{\theta} = \operatorname{argmax}_{\theta} \sum_{i=1}^n \log P(y^{(i)} | x^{(i)})$
- This might fit the training set too well! (including noisy features)
- Poor generalization to the unseen test set — **Overfitting**
- **Regularization** helps prevent overfitting

$$\hat{\theta} = \operatorname{argmax}_{\theta} \sum_{i=1}^n \log P(y^{(i)} | x^{(i)}) - \alpha R(\theta)$$

⁽²⁾
Penalize
large
weights

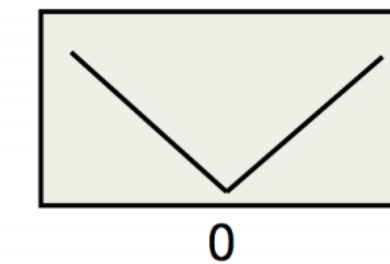
L2 regularization

- $R(\theta) = ||\theta||_2 = \sum_{j=1}^d \theta_j^2$
- **Euclidean distance** of weight vector θ from origin
- L2 regularized objective:

$$\hat{\theta} = \operatorname{argmax}_{\theta} \sum_{i=1}^n \log P(y^{(i)} | x^{(i)}) - \alpha \sum_{j=1}^d \theta_j^2$$

L1 Regularization

- $R(\theta) = ||\theta||_1 = \sum_{j=1}^d |\theta_j|$



- **Manhattan distance** of weight vector θ from origin
- L1 regularized objective:

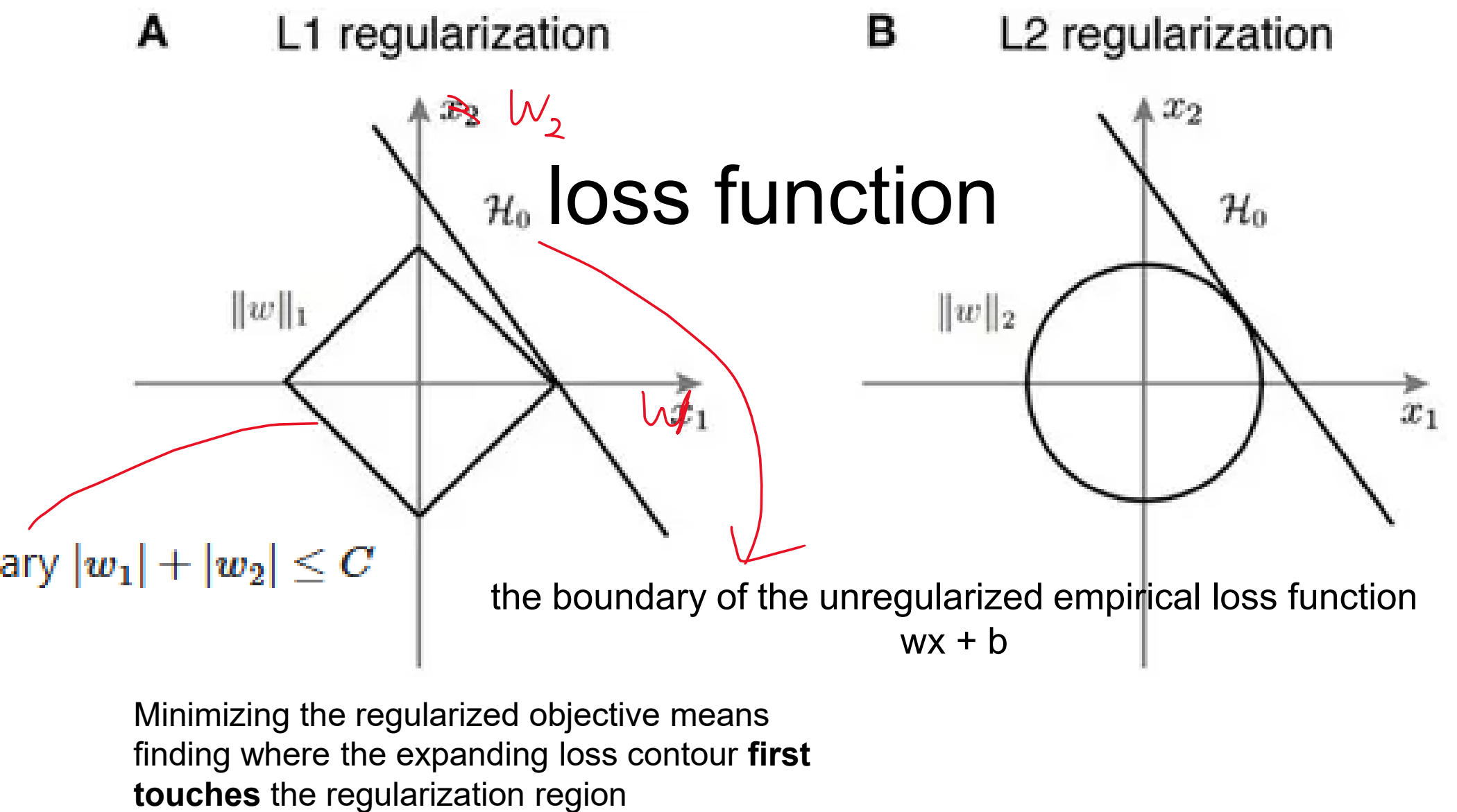
$$\hat{\theta} = \operatorname{argmax}_{\theta} \sum_{i=1}^n \log P(y^{(i)} | x^{(i)}) - \alpha \sum_{j=1}^d |\theta_j|$$

L2 vs L1 regularization

- L2 is easier to optimize - simpler derivation
- L1 is complex since the derivative of $|\theta|$ is not continuous at 0

- L2 leads to many small weights (due to θ^2 term)

- L1 prefers *sparse* weight vectors with many weights set to 0 (i.e. far fewer features used). Why?



$$R(\theta) = ||\theta||_1 = \sum_{j=1}^d |\theta_j|$$

L1 Produces Sparsity: A 1D Walk-Through

$$\min_w J(w) = L(w) + \alpha|w|$$

- If $\frac{\partial L}{\partial w} = -0.3$ (the slope goes down as w increases):

$$\Delta w = -\eta(-0.3) = +\eta(0.3)$$

↳ |

- Taking the negative gradient moves w in the **positive direction** (to the right: $w \rightarrow w + 0.3$).

- **Setup:**

- Weak data signal pushes w rightward: force = $+0.3$
- Regularization strength: $\alpha = 1.0$ (pulls toward 0)

- **Tug-of-War:**

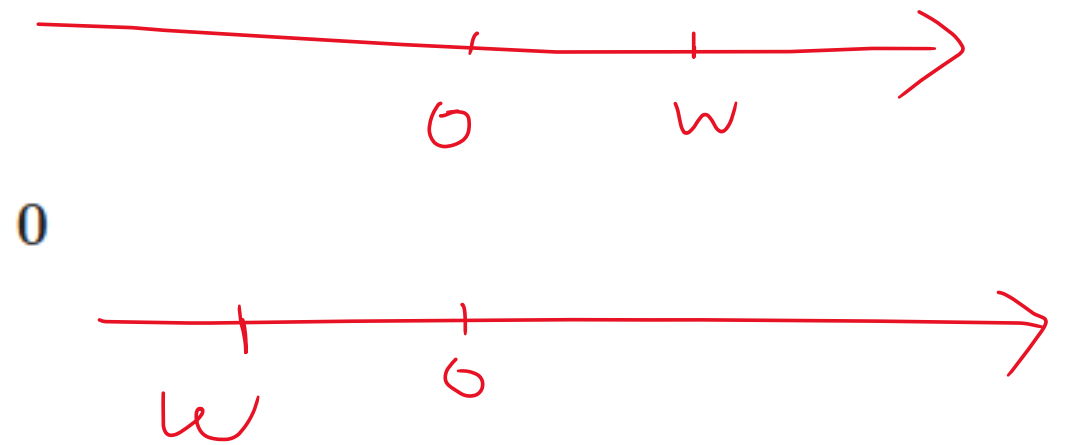
- If $w > 0$: Net force = $0.3 - 1.0 = -\mathbf{0.7} \rightarrow$ pulls left toward 0
- If $w < 0$: Net force = $0.3 - (-1.0) = +\mathbf{1.3} \rightarrow$ pushes right toward 0

- **Trapped at 0 (Subgradient Equilibrium):**

- At $w = 0$, L1 can exert *any* opposing force in $[-\alpha, +\alpha] = [-1.0, +1.0]$
- Since the data signal (0.3) is weaker than α (1.0), it cannot pull w out of zero

- **Core Rule:**

A weight becomes non-zero **only if** $|\text{data force}| > \alpha$. Otherwise, it is clamped strictly to 0 (feature eliminated).



Summary

- Logistic regression - a discriminative classification model that models $P(c|d)$ directly
- Can have more flexible features than Naive Bayes
- Use optimizer to learn parameters
 - For logistic regression, maximizing the likelihood is the equivalent to minimizing the cross entropy loss
 - Likelihood function is convex, but MLE does not have closed form solution
 - Use gradient descent to determine parameters
- Connection to neural networks